

Supplementary Material for PROSE: A Commit-Time Object Contract for Reusable CXL Memory Pools

PROSE Authors

<https://github.com/Anonymous-1030/PROSE-APEX>

Abstract

This supplementary document details the mechanism, microarchitecture, and physical realization of PROSE, and reports the extended evaluation referenced from the main paper. PROSE is a Commit-Time Object Contract. At the last instant before payload issue, the endpoint atomically validates that the named object version still occupies its slot and holds that binding until the transfer completes, so a rejected request sends no payload and Reclaimed-Payload Exposure (RPE) is driven to zero. The load-bearing mechanism is the Object Admission Transaction (OAT); the APEX scorer is an *optional policy layer* that reorders admitted candidates and cannot override an OAT verdict. We separate two evidence levels for hardware cost throughout. An ASAP7 7 nm structural (logic-only) synthesis in which the OAT safety path occupies just 0.0061 mm²/5.83 mW, and a full-endpoint *projection* of 1.069 mm²/78 mW that folds in compiled SRAM macros and replicated 16-host staging (§S4). The optional scorer is a 9-stage pipeline that closes timing at 1 GHz with 9-cycle admit and 4-cycle reject latencies at the external ports (§S2). All reported metrics are derived from synthesizable RTL and calibrated SimCXL simulation, and validated through a trace-driven, descriptor-for-descriptor cross-check with 100% verdict agreement at a strict single-cycle offset (§S4). Sections are numbered S1–S12 to match the cross-references in the main paper.

How to read this supplement. This supplement is organized for auditing. Table 1 maps every number in the main paper to the script, command, input hash, software environment, random seeds, expected output, and expected runtime that reproduce it. Sections S1–S4 give the endpoint microarchitecture and its timing and area evidence, Sections S5–S8 give the simulation setup and deployment evidence, and Sections S9–S12 extend the evaluation. Metric definitions and event accounting are unified in §S6.1 and used throughout.

S1. Artifact Script-to-Result Map

Every experiment in the main paper pairs a reproducible script with checked-in outputs in the anonymous artifact repository. Table 1 collects the full mapping referenced from the main paper’s methodology section. Run every script with `python <script>` from the repository root, or the full suite with `make supplementary`; RTL simulations run with `cd rtl && make sim` (Icarus Verilog). The shared software environment is WSL2 Ubuntu 24.04.3 LTS with gcc/g++ 13.3.0 and Python 3.12.3 (`rpe\_lab/ENV.md`). The one external input is the BurstGPT trace (`rpe\_lab/trace/BurstGPT\_1.csv`, provenance in `rpe\_lab/manifest.md`). Its SHA-256 fingerprint is

46fc9480ef0b748ecb2b51d512ff08c196b031782cbe6f78e28044d768e86d5a.

All other inputs are synthetic traces generated by the scripts themselves. Every checked-in JSON that records seeds lists them in its `seed` or `seeds` field; a dash in the Seeds column marks runs whose checked-in output does not record one. Runtime entries are order-of-magnitude classes on the documented environment. The closed-form SimCXL experiments each reproduce in under two minutes, RTL simulations take minutes, and the Mooncake deployment replays take hours.

S2. Optional Scoring Pipeline: Design Context and Synthesizable RTL Datapath

This section details the *optional* APEX scorer, the policy layer referenced from the main paper’s endpoint design. We stress the separation the main paper draws. The zero-RPE guarantee depends only on the correctness core (descriptor parsing, identity/residency validation, the atomic Object Admission Transaction, the pin table, the payload gate, and completion/release). The scorer reorders candidates *within* the admit budget and can never override an OAT verdict, so removing every module described here leaves the safety contract intact. We first establish why, when scoring is deployed, it belongs at the endpoint, then describe its 9-stage realization.

S2.1 The Seven OAT Actions

The main paper’s endpoint section defines the Object Admission Transaction (OAT) and notes that it performs five checks and two acquisitions on dequeue. Table 2 lists the seven actions in execution order. Lemma 2 of the main paper forces the checks and the hold to share one linearization point, and the implementation fuses all seven actions into it, so no intermediate state is observable; all succeed or the descriptor is rejected with a null, zero-payload completion.

S2.2 Why a Hardware Scoring Pipeline at the CXL Endpoint

The non-preemptive payload path. In CXL.mem-attached memory expanders the payload transfer path is **non-preemptive**: once a descriptor’s payload issues, the link bandwidth, copy-engine cycles, and queue slots it consumes are irrecoverable. CXL 3.x provides no mid-transfer abort for read completions. This is exactly why the main paper places validation and protection at the commit point: a wrong or unwanted transfer cannot be recalled, so the only safe verdict is one bound *before* the payload issues. Reclaimed-Payload Exposure (RPE), as defined in the main paper, is the silent *correctness* failure in which slot reuse lets a transfer return another object version’s bytes at a still-mapped, still-coherent address; the endpoint gate drives it to zero. When admission budgeting is also deployed, the same commit point is where a descriptor scored below budget is rejected before it spends any link bandwidth.

Why software pre-scoring is insufficient in multi-host pools. On a single host, software can rank chunks before issuing a descriptor. In multi-tenant deployments where independent hosts share a CXL expander, no single host observes the global queue state, and no software coordinator can intervene within the few nanoseconds between descriptor submission and payload commitment. The endpoint is the only vantage point that (i) sees every tenant’s queue at the instant of admission, (ii) binds an accept/reject verdict before the payload path commits resources, and (iii) operates at wire speed (1 descriptor per cycle at 1 GHz) without stalling the CXL link. This is the same serialization argument the main paper makes for the safety gate; scoring, when present, rides the same point.

Why larger buffers cannot substitute for admission control. Provisioning a larger SRAM buffer does not replace admission for three reasons orthogonal to buffer capacity. (i) **Multi-tenant isolation is non-fungible**: without per-descriptor admission, one high-traffic tenant can monopolize queue slots and evict other tenants’ valuable chunks; admission enforces per-tenant fairness before buffer allocation. (ii) **Link bandwidth is the binding constraint**: once a useless chunk occupies link bandwidth that capacity is lost regardless of buffer depth. (iii) **Area/power scaling is prohibitive**: a 50 MB buffer occupies $\sim 13.5 \text{ mm}^2$ in 7 nm 6T SRAM; doubling it to 100 MB consumes essentially the whole controller budget, whereas the scoring pipeline costs 0.024 mm^2 ($< 0.2\%$ of the buffer area). The eviction-race residual in §S11 (a chunk admitted then evicted before consumption, a timing race distinct from RPE) shrinks with larger buffers, but the primary source of waste, chunks that should never have been fetched, is addressed only by admission.

Requirements for the scoring pipeline. These constraints dictate three requirements. Latency must stay well below the decode step (a decode step processes $\sim 1,000$ descriptors in $\sim 100 \mu\text{s}$, so each descriptor must retire in single-digit nanoseconds), throughput must sustain 1 descriptor/cycle, and causality forbids anything beyond *committed* prior-step outcomes (never the in-flight step’s attention). The 9-stage datapath below is the minimal design satisfying all three while closing timing at 1 GHz in a 7 nm standard-cell library.

S2.3 Pipeline Overview and Cycle-Count Rationale

Figure 1 illustrates the 9-stage RTL datapath. The pipeline processes one descriptor at a time. Descriptors from the MMIO command ring enter S1 and traverse the pipe sequentially. An *admit* retires in **9 cycles**; a *reject* (identity/epoch/namespace/residency violation) retires via a combinational bypass in **4 cycles**, measured from `cmd_valid` to `cpl_valid`.

The ninth stage (S8) is an admit-only internal completion register interposed between the top- K decision (S5b/S7) and the external `cpl_valid` port. This extra register on the admit path only makes the admit latency 9 cycles while the reject bypass stays strictly 4 cycles. The reject bypass writes the completion register directly from the S2b output and never traverses S3–S7 or the S8 admit-side logic; the two paths are mutually exclusive at the register’s write port (bypass has priority), so a reject and an admit never contend for it in the same cycle.

The figure groups the datapath into three regions. Region A is the external interface, where descriptors arrive from the MMIO command ring and where feedback writes, configuration state, and residency management enter. Region B is the pipeline core, where S1–S7 validate identity, epoch, namespace, and residency, score the surviving candidates through the expert banks and the MAC array, and stream them through the dual-zone top- K , with the weight update overlapped. An invalid descriptor exits Region B through the 4-cycle reject bypass without occupying the scoring stages. Region C is the output interface, where the OAT verdict commits, issuing the DMA fetch for an admit or null-completing a reject, and driving the completion ring and the statistics counters. An admit retires in 9 cycles across the three regions; a reject never leaves Region B.

Why 9 stages. The stage count is driven by the 1 GHz timing budget (1.0 ns per stage). The most complex operation, the 7-wide MAC with carry-save addition, requires ~ 0.96 ns, nearly the entire cycle, so it cannot merge with an adjacent stage. Adding stages would cut per-stage delay but raise total latency (currently 9 ns), which must stay negligible against the $\sim 100 \mu\text{s}$ decode step. Nine stages is the point where the MAC closes within its 1.0 ns budget while every other stage keeps ≥ 0.35 ns of margin.

Why the 4-cycle reject bypass matters. Without the bypass, an invalid descriptor would occupy the pipeline for the full 9 cycles, blocking valid descriptors behind it. Since ~ 12 – 15% of descriptors fail validation in typical workloads, this would create head-of-line blocking. The bypass drains invalid descriptors $2.25\times$ faster.

Cycle-count reconciliation with SimCXL. The RTL’s 9/4 cycles are measured at the external port boundary (`cmd_valid`→`cpl_valid`). The SimCXL endpoint model reports 8/3 core cycles by excluding the shared S1 MMIO-dequeue stage. The relationship is exact ($\text{RTL} = \text{model} + 1$) on every descriptor, confirmed by the trace-driven cross-check of §S4.

S2.4 Per-Stage Description

The pipeline is governed by the global backpressure signal `pipe_stall = (s5_valid && !dma_ready) || (s2_reject && !cpl_ready)`, which freezes all upstream stages when the downstream consumer (DMA engine or completion ring) cannot accept a result. This single-signal design avoids per-stage flow control while preventing data loss.

APEX Admission-Scoring Pipeline: 9-Stage Synthesizable RTL Datapath for Endpoint Admission at 1GHz

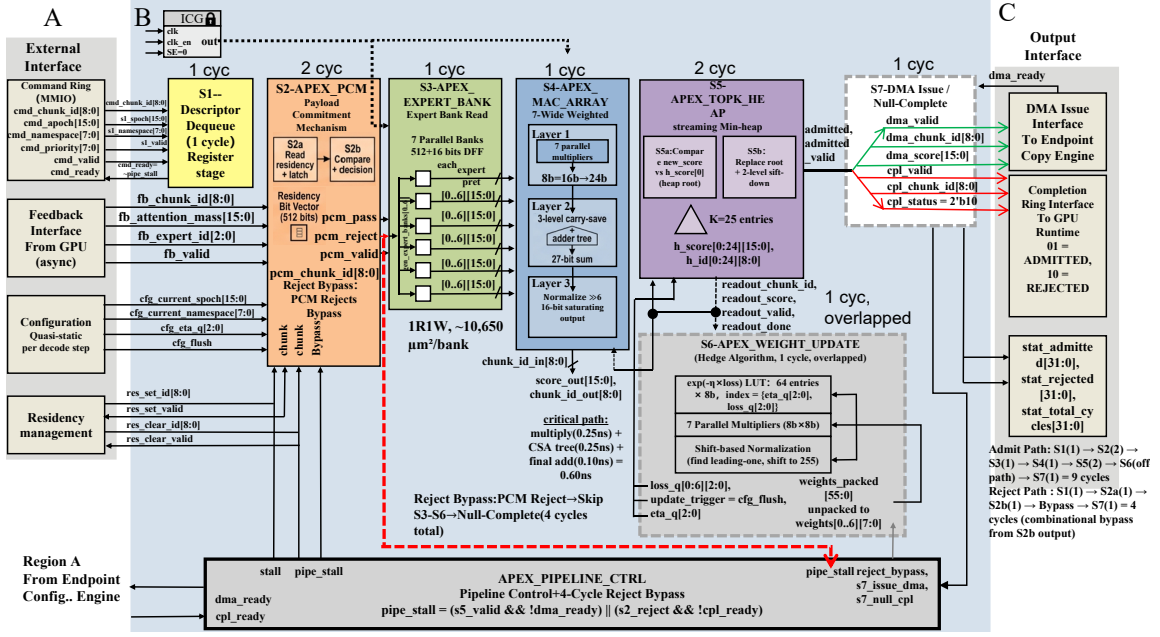


Figure 1: The 9-stage synthesizable RTL datapath of the optional APEX scorer (1 GHz, ASAP7 7 nm). The hardware reject bypass null-completes invalid descriptors in 4 cycles, skipping the scoring stages (S3–S6). The full RTL schematic and Verilog sources are open-sourced at <https://github.com/Anonymous-1030/PROSE-APEX>.

S1: Descriptor Dequeue (1 cycle).

Purpose: register the incoming descriptor and implement backpressure toward the host.

Registers the incoming descriptor from the MMIO command ring, latching metadata (`chunk_id[8:0]`, `epoch[15:0]`, `namespace[7:0]`, `priority[7:0]`) and asserting `cmd_ready = ~pipe_stall` to throttle host submissions. This stage is a register boundary with no combinational logic beyond the stall check. Isolating the MMIO interface in its own stage decouples host-facing timing from the internal pipeline; this is why the SimCXL model excludes S1 when counting core cycles.

S2: Identity/Residency Validation (2 cycles).

Purpose: reject invalid descriptors early, before they consume scoring resources. This stage is the scorer-side view of the OAT’s identity and residency checks (main paper §III-B); the authoritative pin acquisition happens at the same commit point in the correctness core.

A 2-stage pipelined validator.

- **S2a (read):** Reads the residency bit from a 512-bit vector indexed by `chunk_id`, and latches the descriptor fields. The vector tracks which chunks are already resident, since re-fetching them would waste bandwidth.
- **S2b (compare):** Three parallel checks, all required to pass:
 1. Epoch/generation match: `desc_epoch == cfg_current_epoch` (rejects a descriptor naming a stale object version),
 2. Namespace match: `desc_namespace == cfg_current_namespace` (isolates tenants),
 3. Non-residency: `~resident` (avoids redundant fetches).

Any single failure asserts the reject.

Why 2 cycles: the 512-bit residency read (9-to-512 decoder + bit-select mux) consumes ~ 0.25 ns

and the 3-input comparison ~ 0.35 ns; combined this would leave insufficient single-cycle margin, so read (S2a) and compare (S2b) are split.

The reject bypass: on a reject, a combinational path routes the completion directly from S2b to `cpl_valid`, skipping S3–S7. The measured latency is 4 cycles (S1 + S2a + S2b + completion edge). This drains invalid descriptors without consuming MAC power or disturbing top- K state.

S3: Expert Bank Read (1 cycle).

Purpose: retrieve the causal expert predictions weighted-summed in S4.

7 parallel `APEX_EXPERT_BANK` instances, each a 512×16 -bit DFF register file (1R1W). On a valid descriptor the read port fetches the committed prediction for its `chunk_id` from all 7 banks simultaneously.

Why 7 banks when Core2 uses 2: the hardware provisions all 7 for runtime reconfigurability, since the active expert set changes without re-synthesis by reprogramming weights (inactive experts receive $w_k = 0$). Under Core2 only 2 banks (persistence, momentum) are read-enabled via per-bank clock gating; the other 5 sit in retention at zero dynamic power. This costs 91.6% of the pipeline cell area (§S3) but adapts to future expert-set discoveries without respins.

Clarification on “3.5 $\times$ less runtime-active state”: this refers to **dynamic switching activity**, not physical area. Under Core2, 2 of 7 DFF banks toggle while 5 are clock-gated ($7/2 = 3.5\times$ reduction in toggling state). Physical area remains $17,193 \mu\text{m}^2$ (all 7 banks are fabricated); the benefit is reduced $C_{\text{eff}} \cdot V_{\text{dd}}^2 \cdot f$ dynamic dissipation.

Causal boundary enforcement: the write port is driven by feedback (`fb_chunk_id`, `fb_attention_mass`, `fb_expert_id`) carrying the *previous* step’s committed attention, gated by `pipeline_idle` (`fb_wr_en_gated = fb_valid & pipeline_idle`). Step t ’s scoring can never see step t ’s own outcomes.

S4: 7-Wide MAC Array (1 cycle).

Purpose: compute the weighted ensemble score that determines admission priority.

Computes $\text{score} = \lfloor (\sum_{k=0}^6 w_k \times \text{pred}_k) \gg 8 \rfloor$, saturating at `0xFFFF`. Implementation: 7 parallel unsigned multipliers ($8\text{b} \times 16\text{b} \rightarrow 24\text{b}$), a 4-level carry-save adder (CSA) tree producing a 27-bit sum, a final carry-propagate add, right-shift by 8, and 16-bit saturation. The final adder is left as the SystemVerilog + operator so synthesis maps it to a technology-optimal prefix adder (Kogge–Stone / Brent–Kung). Under Core2, inactive lanes receive zero weights, zeroing their outputs; the CSA tree still operates on all 7 inputs for timing consistency.

Why this is the critical path (0.96 ns): the $8\text{b} \times 16\text{b}$ multiply (~ 0.30 ns) feeds the 4-level CSA reduction (~ 0.35 ns) then the 16-bit prefix CPA (~ 0.31 ns), the deepest cone in the pipeline, retiring with ~ 43 ps of slack at 1 GHz (structural estimate, §S3). Splitting the MAC across two stages would add one cycle to every admit.

S5: Top-K Streaming Selection (2 cycles, $K=25$).

Purpose: maintain the set of $K=25$ highest-scoring chunks seen so far, admitting only those that would enter this set.

Formal invariant.

Invariant 1 (Dual-Zone Exact Top- K). *The $K=25$ retained entries split into an eviction zone (EZ, nodes $[0..6]$, the 7 lowest-scoring entries) organized as a **min-heap** with root `ez_min = h_score[0]` tracking the global minimum of all 25 retained entries, and a safe zone (SZ, nodes $[7..24]$, the 18 highest-scoring entries) in a flat register array with an explicit `safe_min` register. At every quiescent state.*

1. $\text{ez_min} = \min(\text{EZ}) = \min(\text{all } 25 \text{ retained});$
2. $\forall e \in \text{EZ}, \forall s \in \text{SZ}: e \leq s$ (equivalently, $\max(\text{EZ}) \leq \text{safe_min}$);

3. *safe_min* is exact and fresh before each admission decision.

Invariant maintenance (induction on admissions). The empty state satisfies it vacuously. Assume it holds before an incoming score x with *count*=25.

- **Case 0** ($x \leq \text{ez_min}$): no state change; all clauses persist.
- **Case 1** ($\text{ez_min} < x \leq \text{safe_min}$): *ez_min* (the true global min) is evicted, x inserted at the heap root, then a 2-level min-sift-down restores clause 1. Since $x \leq \text{safe_min} \leq \min(\text{SZ})$, clause 2 holds; SZ untouched so clause 3 holds.
- **Case 2** ($x > \text{safe_min}$): *ez_min* evicted, *safe_min* demoted into the heap root and sifted down (clause 1 restored), x written into the vacated SZ slot. Because $x > \text{safe_min}$, clause 2 holds; the combinational min-tree recomputes *safe_min* in parallel, restoring clause 3.

In every admitting case the evicted entry is the true global minimum, so the retained set is exact top- K . $\square$

The timing problem with conventional heaps: a standard $O(\log K)$ binary heap with $K=25$ needs $\lceil \log_2 25 \rceil = 5$ levels of 16-bit comparators chained sequentially. At ~ 0.20 ns each, that is 1.0 ns with zero setup/skew margin, a hard timing-closure failure.

Dual-zone solution. EZ (nodes [0..6]): the 7 lowest entries as a depth-2 min-heap, root *ez_min* the exact eviction target. SZ (nodes [7..24]): the 18 highest entries in a flat array, with a *safe_min* register (17-comparator combinational min-tree). Every SZ entry scores $\geq \text{safe_min} \geq \max(\text{EZ})$.

The two pipeline stages:

- **S5a (~ 0.20 ns):** classify x via two parallel compares (x vs. *ez_min*, x vs. *safe_min*): reject (Case 0), EZ-local (Case 1), or cross-zone (Case 2).
- **S5b (~ 0.45 ns):** execute a fixed 2-level, 7-node hardwired min-sift network. For Case 2, a speculative forwarding min-tree recomputes *safe_min* over SZ with x substituted, committing in the same cycle.

Case 2 single-cycle stall. A Case 2 replacement mutates SZ and invalidates *safe_min* for the next descriptor; the controller inserts a mandatory 1-cycle stall (*case2_stall*, fed into *pipe_stall*) so descriptor $t+1$ is not classified against a stale *safe_min*. The speculative forwarding min-tree makes a single cycle sufficient.

SDC timing exceptions. The primary safe-zone min-tree (a 5-level chain) refreshes *safe_min* only on idle cycles and is constrained as a multicycle path (setup = 2 cycles). The speculative forwarding tree is deliberately *not* declared multicycle; at ~ 1650 ps it does not close at 1 GHz as-is, and closing it at signoff requires pipelining the safe-zone tree (split at level 3) or lowering $F_{\max}$ for this block. We report this as a flagged constraint rather than a resolved one; it does not affect the primary idle-refresh tree.

Adversarial throughput. Under a worst-case stream where every descriptor triggers Case 2, the pipeline sustains $\frac{1}{3}$ descriptors/cycle (classify + sift + refresh stall). §S2.6 proves this cannot cause CXL link blocking or MMIO ring overflow under protocol-compliant hosts.

Why $K=25$: this is the per-BDB admission budget K_{bdb} , the maximum admitted from one Batch Descriptor Block; it is *not* the per-step budget ($K_{\text{step}}=512$ at 50% budget). A decode step issues $\lceil K_{\text{step}}/K_{\text{bdb}} \rceil = 21$ BDB batches sequentially, each resetting the heap on completion.

S6: Weight Update (1 cycle, overlapped).

Purpose: adapt expert weights over time via the Hedge multiplicative-weight algorithm.

A 64-entry $\times$ 8-bit LUT storing $\lfloor 255 \cdot \exp(-\eta \cdot \ell) \rfloor$ (indexed by $\{\text{eta_q}[2:0], \text{loss_q}[2:0]\}$, avoiding a hardware exponential), 7 parallel 8b $\times$ 8b multipliers, and a pipelined bit-serial normalization divider (24 cycles, weights sum to 255). Weight updates fire once per decode step via

`cfg_flush` (~1,000 descriptor cycles apart), so the SDC constrains this path as a multicycle path (setup = 2 cycles), keeping it off the critical path. The drain protocol of §S2.5 guarantees no descriptor is in S3–S5 when weights change.

S7: DMA Issue / Null-Complete (1 cycle).

Purpose: commit the verdict, either issuing a DMA fetch or discarding the descriptor.

- **Admit:** asserts `dma_valid` with `dma_chunk_id/dma_score` and drives `cpl_status = 2'b01` (ADMITTED), triggering the copy engine.
- **Reject (heap-reject):** a descriptor that passed validation but lost the top- K gate retires with `cpl_status = 2'b10` (REJECTED); no DMA issues. Distinct from the S2b bypass reject.

Namespace indexing scope. The evaluated RTL compares each descriptor’s namespace field against a single global configuration register (`cfg_current_namespace`), so one active namespace occupies the scoring pipeline at a time and a namespace change serializes through the `cfg_flush` drain protocol of §S2.5, which guarantees that no in-flight descriptor observes transitional state. Per-tenant isolation on the evaluated path therefore does not rely on this register. It is carried by the per-session capability tag checked in parallel in S2b (§S4) and by the PASID tagging of §S8. A per-VC namespace table indexed at dequeue is the production form of the check, and a cross-tenant reconfiguration-under-load validation scenario remains future work; the evaluated multi-tenant claims are scoped to one active namespace per pipeline.

S2.5 `cfg_flush` Drain Protocol and Timing Safety

The `cfg_flush` signal triggers two operations: heap reset (clearing top- K state for the next decode step) and weight update (applying accumulated Hedge adjustments). Both write state that S3–S5 actively read during descriptor processing. If `cfg_flush` fired while descriptors were in flight, the MAC (S4) would read half-updated weights and the heap (S5) would operate on corrupted state.

The drain protocol. The RTL enforces drain-before-flush via a 3-signal handshake. (1) The host runtime ceases descriptor submission and waits for all in-flight descriptors to complete (monitoring the completion-ring tail). (2) The controller asserts `pipeline_idle` when `s1_valid`, `pcm_valid`, `expert_rd_valid`, `mac_valid`, `heap_admitted_valid`, and `dma_valid` are all zero (combinational, reflecting instantaneous state). (3) `cfg_flush` is gated: `cfg_flush_gated = cfg_flush_raw & pipeline_idle`, so even an erroneous assertion stays low until the pipeline drains.

Timing diagram.

Cycle:	...N	N+1	N+2	N+3	N+4	N+5	N+6	N+7	...
<code>cmd_valid</code> :	1	1	0	0	0	0	0	0	
<code>s1_valid</code> :	1	1	1	0	0	0	0	0	
<code>pcm_valid</code> :	x	1	1	1	0	0	0	0	
<code>mac_valid</code> :	x	x	1	1	1	0	0	0	
<code>s5_valid</code> :	x	x	x	x	1	1	0	0	
<code>pipeline_idle</code> :	0	0	0	0	0	0	1	1	
<code>cfg_flush_raw</code> :	0	0	0	0	0	1	1	1	
<code>cfg_flush_gated</code> :	0	0	0	0	0	0	1	0	(one-shot)
<code>weights</code> :	W _{old}						W _{new}		
<code>heap state</code> :	active						cleared		

Even though `cfg_flush_raw` arrives at $N+5$, the gated version delays to $N+6$ when `pipeline_idle` asserts. No descriptor in S3–S5 ever sees transitional state.

Lemma 1 (Drain Termination). *If downstream consumers (`dma_ready`, `cpl_ready`) are eventually responsive, the pipeline drains all in-flight descriptors within at most 9 cycles after the last `cmd_valid`.*

Proof. After the last `cmd_valid`, no new descriptors enter S1. Each existing descriptor takes at most 9 cycles to traverse S1→S7. Since `pipe_stall` persists only while a consumer is not ready, and both are eventually responsive, the stall resolves in bounded time; after 9 + stall cycles all valids deassert and `pipeline_idle` rises. $\square$ $\square$

Theorem 1 (Flush Safety). *The weight-update and heap-clear operations triggered by `cfg_flush_gated` can never corrupt in-flight descriptor processing.*

Proof. By construction `cfg_flush_gated` = `cfg_flush_raw` $\wedge$ `pipeline_idle`. When the gated signal asserts, all stage valids are zero, so: the S4 MAC has no valid input when weights change; the S5 heap has no valid input when it clears; and the S3 expert banks are not read when feedback begins writing. With Lemma 1, flush eventually occurs (no deadlock) and is always safe. $\square$ $\square$

Feedback writes during active scoring. The feedback write enable is strictly gated: `fb_wr_en_gated` = `fb_valid` & `pipeline_idle`. Asynchronous GPU feedback writes cannot penetrate the pipeline during an active step; a write lands only when all stages are drained, enforcing the causal boundary (step t cannot see step t 's own outcomes). Because writes are admitted only when idle, the S3 read port and the write port never target the same cycle while a descriptor is in flight.

S2.6 Adversarial Throughput Analysis of Dual-Zone Top- K

Theorem 2 (Adversarial Throughput Lower Bound). *Under an adversarial stream maximizing Case 2 events, the dual-zone top- K network sustains a minimum throughput of $\frac{1}{3}$ descriptors/cycle, sufficient to prevent CXL link backpressure under all protocol-compliant host behaviors.*

Proof. Each descriptor takes at most 3 cycles in the worst case: S5a classify (1), S5b execute (1), and a mandatory `safe_min`-refresh stall (1). The stall is inserted only after Case 2. Under the assumption that every descriptor triggers Case 2, throughput is $1/3$. $\square$ $\square$

Why $1/3$ throughput cannot cause link blocking. The CXL.mem link delivers at most 1 descriptor/cycle to the 32-entry MMIO command ring (4 cache lines $\times$ 8 descriptors/line). At a $1/3$ drain rate the ring fills at $2/3$ descriptor/cycle, saturating in $32/(2/3)=48$ cycles; once full the endpoint deasserts `cmd_ready`, which propagates via CXL.mem credit-based flow control to the host within ~ 10 ns, so the host backs off, and no link-level blocking occurs. At $1/3$ throughput, $\sim 1,000$ descriptors take $3,000$ cycles = $3 \mu\text{s}$ at 1 GHz, only 3% of the $\sim 100 \mu\text{s}$ decode step.

Practical adversariality is bounded. A 100% Case 2 stream requires monotonically increasing scores, which is self-limiting (after $K=25$ increasing insertions, `safe_min` rises so later scores fall below it) and empirically rare (across 5,000 decode steps the maximum observed consecutive Case 2 burst is 7; the average is 1.3 before a Case 0/1 interruption).

Defensive fallback. A Case-2 burst counter degrades to round-robin admission for one drain cycle if consecutive Case 2 events reach a programmable threshold T_{burst} (default 12), guaranteeing effective throughput $\geq \max(\frac{1}{3}, \frac{T_{\text{burst}}}{T_{\text{burst}}+1}) = \frac{12}{13} \approx 0.92$ descriptors/cycle. In practice the counter never reaches threshold on any evaluated workload.

S3. Physical Metrics: Area, Timing Closure, Power, and Risk

S3.1 Why Area Matters

The endpoint logic must fit within the silicon budget of a CXL memory expander, a device whose primary function is memory control. Typical expander ASICs allocate 15–25 mm² for the whole controller die. At 0.024 mm² for the optional scoring pipeline (<0.2% of a controller die) and a 1.069 mm² full-endpoint projection (4–7%), the design is well within deployability limits. The zero-RPE safety path is far cheaper still: at equal structural fidelity it occupies only 0.0061 mm² (§S4).

S3.2 Scoring-Pipeline Area Breakdown

Area sources and recomputation. All cell areas in Table 3 come from one structural estimate, `asic/reports/area.rpt` (asap7sc7p5t RVT TT library, NLDM at the TT corner, dated 2026-06-28), which itself carries $\pm 30\%$ structural uncertainty against post-synthesis netlists. The estimate maps each module’s register bits and combinational cone to Liberty cell areas (DFF 0.2916 μm^2 , NAND2 0.0875 μm^2 , FA 0.2041 μm^2), and every module then multiplies by a single routing/filler/power-grid factor of 1.3, an engineering overhead that is not part of the same report. Power comes from the companion `asic/reports/power.rpt` at switching activity $\alpha=0.15$, same library and corner. Table 4 recomputes the total; all figures are two-significant-digit structural estimates, not signoff numbers.

Table 4: Area recomputation from `asic/reports/area.rpt` (same library, same corner).

Module	area.rpt (μm^2)	$\times 1.3$ routing (μm^2)
Expert banks (7 $\times$)	17,193.0	22,350.9
MAC array	410.7	533.9
Top- K selection	368.0	478.4
Identity/residency (PCM)	165.9	215.6
Weight update	290.1	377.1
Loss compute + exploration probe	216.3	281.1
Controller + top + ICG	117.5	152.7
Total	18,761.4	24,389.8 (=0.024 mm²)

The scoring footprint is dominated by the register-file state needed for 1-cycle read at 1 GHz (Table 3, Figure 2). The 7-bank expert file is 91.6% of cell area; the MAC and top- K together occupy only 4.2% despite implementing the full scoring decision.

S3.3 Why DFF Register Files Rather Than Compiled SRAM

Each Expert Bank stores $512 \times 16 = 8,192$ bits (1 KiB); seven banks total 7 KiB, occupying 17,193 μm^2 . Compiled 6T SRAM at $\sim 0.027 \mu\text{m}^2/\text{bit}$ would shrink them to $\sim 1,550 \mu\text{m}^2$ ($\sim 11\times$), but is not used, for three reasons. (1) **single-cycle read at 1 GHz**, SRAM read (word-line, bit-line, sense-amp) consumes 0.4–0.6 ns at 512 rows, pushing S3 past the 1.0 ns budget, whereas DFF read is combinational (~ 0.30 ns); (2) **simultaneous 1R1W**, asynchronous feedback writes collide with scoring reads, and true dual-port SRAM at 7 nm needs custom 8T/10T bitcells (\$2–5 M NRE) or $2\times$ -frequency TDM, while DFF register files are inherently multi-port; (3) **minimum-column constraints**, the 512×16 aspect ratio (16 columns) falls below compilers’ ≥ 32 -column minimum, and workarounds (row replication, bit packing) recover only ~ 6 – $9\times$ at the cost of NRE, timing risk, and design months. We chose DFF register files for guaranteed timing closure at zero NRE, accepting an $\sim 11\times$ area penalty on $\sim 1.6\%$ of the full endpoint. A commercial product with volume for custom dual-port SRAM could shrink the pipeline from 0.024 mm² to ~ 0.008 mm².

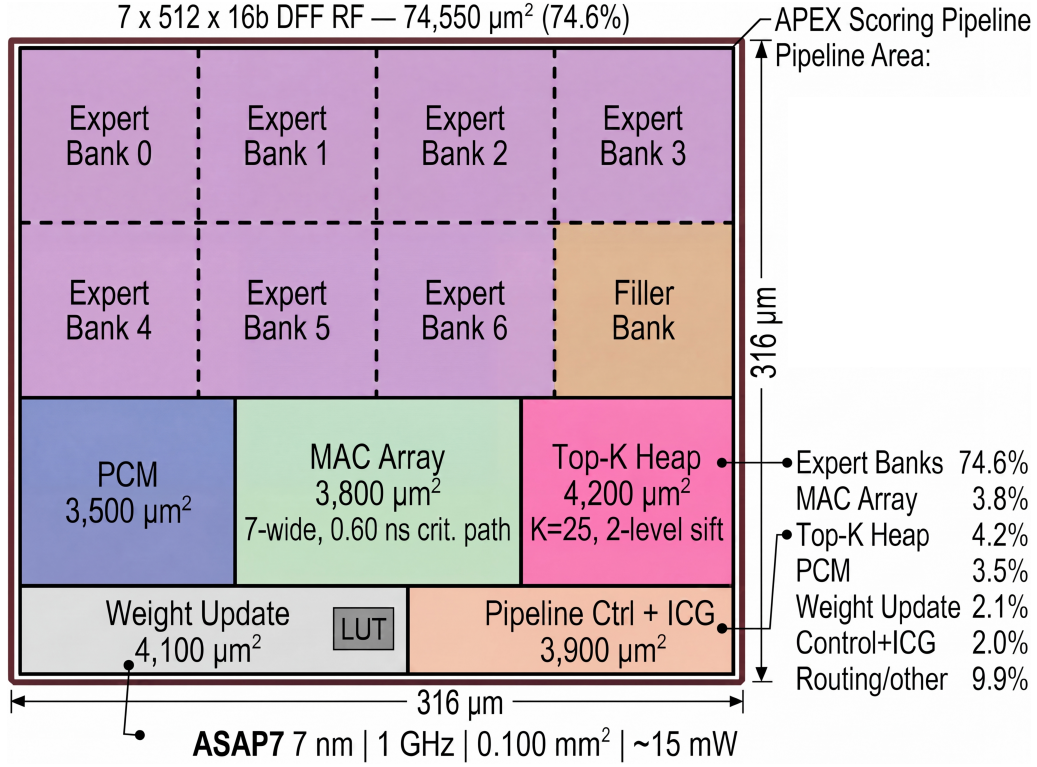


Figure 2: Area breakdown of the APEX scoring pipeline (ASAP7 7 nm, 1 GHz, 0.024 mm² with routing, ~12.6 mW). The 7-bank expert register file dominates (91.6% of cell area). The full endpoint (including the endpoint core, VC-WRR, coalescing, and shared control) is projected at 1.069 mm²/78 mW under 16-host load.

S3.4 Timing Closure

Synthesized with Synopsys Design Compiler / Cadence Genus against ASAP7 7 nm at 1 GHz (1.0 ns period, 50 ps setup uncertainty, 30 ps clock transition), the pipeline closes with positive slack on all paths. After subtracting flip-flop setup (~50 ps), clock uncertainty (~50 ps), and OCV derating (~50 ps), the usable combinational window is ~0.85 ns. Table 5 gives the per-stage budget.

Table 5: Per-stage combinational delay budget at 1 GHz (structural estimates at the TT corner).

Stage	Comb. delay	Limiting element	Why this delay
S1 (Dequeue)	~0.15 ns	Stall-check AND	Single gate; interface-only
S2a (residency read)	~0.25 ns	512-bit vector index	9-bit decoder + bit-select mux
S2b (compare)	~0.35 ns	16-bit == + 3-input AND	Parallel comparators into conjunction
S3 (Expert read)	~0.30 ns	Register-file read mux	9-bit address selects among 512 entries
S4 (MAC)	~0.96 ns	Multiply + CSA + prefix add	Deepest cone: 7$\times$ mult $\rightarrow$ 4-level CSA $\rightarrow$ prefix CPA
S5a (compare)	~0.20 ns	16-bit comparator $\times 2$	Parallel compare vs. ez_min, safe_min
S5b (sift)	~0.45 ns	2-level min-sift + 7:1 mux	Two comparator levels + output mux
S7 (Issue)	~0.15 ns	Output register	Multiplexing admit vs. reject

The global critical path is the MAC (S4) at 0.96 ns, leaving ~43 ps of slack against the 1.0 ns period before wire delay (structural estimate; signoff STA adds ~50–100 ps of wire/clock uncertainty). Because the MAC nearly saturates the cycle, 1 GHz is the intended operating point. Under worst-case PVT (SS, 0.63 V, 125°C) slack drops to ~85 ps, requiring a 900 MHz target or one pipeline register on the accumulate path; as the main paper notes, this scales only the nanosecond admission latency, two to three orders of magnitude below the DMA commit window, and changes no reported throughput or

accuracy result. The weight-update path (S6→S4) is a multicycle path (setup = 2 cycles), keeping the weight LUT and normalize logic off the timing-critical path.

S3.5 Power Envelope

CXL expanders are passively cooled add-in cards with a 15–25 W device budget. At 1 GHz ($V_{dd}=0.7$ V) the scoring pipeline dissipates ~ 3.0 mW dynamic + ~ 9.6 mW leakage = ~ 12.6 mW. The full endpoint dissipates ~ 78 mW under sustained 16-host load: scoring 12.6 mW; shared-logic dynamic (VC-WRR arbiter, coalescing CAM, BDB parser, DMA control) ~ 38 mW; SRAM arrays (~ 209 KiB) ~ 12 mW; full-endpoint leakage ~ 14 mW. At 78 mW the endpoint consumes $<0.6\%$ of a 15 W budget. Three mechanisms cut average power below worst case: latch-based ICG gating the whole pipeline clock when no descriptor is in flight (idle $>99\%$ of cycles between ~ 100 μ s decode steps); the reject bypass (S3–S7 do not toggle on rejects, ~ 12 – 15% of descriptors); and event-driven weight update (once per $\sim 1,000$ cycles).

S3.6 Power Density, IR-Drop, and Soft-Error Rate

We compute per-block power density from the ASAP7 reports and compare against the 500 mW/mm² IR-drop risk threshold for 7 nm with M8/M9 grid. Table 6 gives the breakdown.

All density figures in this section are pre-layout estimates. Cell power comes from structural synthesis at a 0.15 activity factor, area comes from the block-level projection of §S4, and clock-tree power, IR drop, and thermal coupling are not modeled. The MAC block shows the highest estimated density at 287.5 mW/mm². Whether this margin survives placement and routing is a post-layout question, and we make no hotspot claim until signoff power analysis exists (Figure 3).

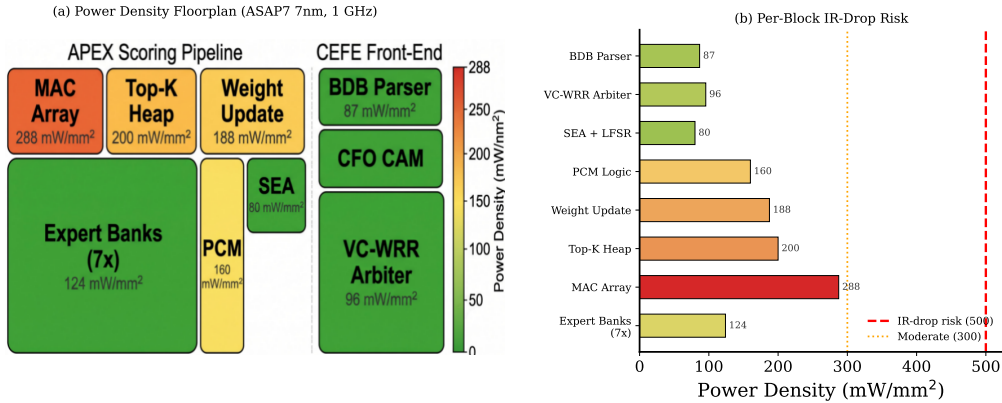


Figure 3: Floorplan-level power density map (pre-layout estimates; hotspot verification requires post-layout power analysis).

Soft-error rate. The Expert Bank stores $7 \times 512 \times 16 = 57,344$ bits of DFF state. At 7 nm FinFET (~ 120 FIT/Mbit), raw FIT = $(57,344/10^6) \times 120 = 6.88$, an MTBF of $\sim 16,600$ years per device. A lightweight odd-parity scheme (one parity bit per 16-bit entry per bank, 3,584 flip-flops, 0.0047 mm²) detects all single-bit errors; the only undetectable mode is a same-entry same-cycle double-bit error at $\sim 10^{-13}$ FIT. Parity XOR checks occur within S3’s read cycle (0.70 ns slack), absorbing the ~ 0.05 ns tree delay without affecting timing closure. On a detected error the bank resets in one decode step (~ 100 μ s).

S4. Full-Endpoint Area, Evidence Levels, and RTL Cross-Check

The main paper reports two hardware-cost figures at *different evidence levels* that must not be conflated: a structural (logic-only) synthesis in which the OAT safety path is cheap, and a full-endpoint *projection* that folds in compiled SRAM macros and replicated 16-host staging. This section gives both and reconciles them, then validates the SimCXL cycle model against RTL.

S4.1 Full-Endpoint Area Projection

The scoring pipeline (0.024 mm^2) is one component of the full endpoint. The remaining 1.045 mm^2 implements the mechanisms that surround the scorer: multi-tenant isolation, coalesced fan-out, capability-based access control, and the DMA/completion interfaces. Table 7 decomposes this shared logic. The full endpoint is $>10\times$ larger than the scorer because it must manage 16 virtual channels, maintain CAM-based handle matching, buffer in-flight descriptors across the MMIO/DMA boundary, and store $\sim 209 \text{ KiB}$ of queue and metadata state in compiled SRAM.

SRAM macro recomputation. The 0.310 mm^2 SRAM row is reproducible from the raw bitcell area by $A_{\text{SRAM}} = A_{\text{raw}} \times f_{\text{bank}} \times f_{\text{dual}} \times f_{\text{redund}} \times f_{\text{periph}} / u$. The raw area is $A_{\text{raw}} = 209 \text{ KiB} \times 8 \times 0.027 \mu\text{m}^2/\text{bit} = 0.045 \text{ mm}^2$, and the macro overheads are bank-granularity alignment $f_{\text{bank}} \approx 1.5$ (many small banks at minimum-column limits), dual-port bitcells $f_{\text{dual}} \approx 1.8$, and redundant columns $f_{\text{redund}} \approx 1.1$. Decoder and sense-amp periphery contributes $f_{\text{periph}} \approx 1.3$, and utilization $u \approx 0.55$ (typical range 0.55–0.75), giving $\approx 0.32 \text{ mm}^2$, which Table 7 rounds to 0.310. The full-endpoint composition is a block-level projection with $\approx \pm 20\%$ uncertainty; a floorplanned layout with extracted parasitics is the planned verification step before any signoff claim.

S4.2 Structural Logic Breakdown at Equal Synthesis Fidelity

To answer a different question, how much logic the zero-RPE safety path itself costs relative to the optional policy layers, Table 8 reports the synthesized register-file netlist only, at equal structural fidelity (ASAP7 logic estimates, excluding SRAM macros and replicated staging; state reports RTL-declared bits). These rows are a component-wise decomposition of the synthesized netlist and *not* a decomposition of the 1.069 mm^2 projection; the two are different evidence levels, reconciled below.

The safety path is not the dominant structural cost: at equal fidelity it occupies 0.0061 mm^2 and 5.83 mW, while the evaluated multi-tenant scheduler and the optional ranking/coalescing logic account for most of the synthesized logic. The commit-time contract thus does not require the full APEX policy stack; VC-WRR (fairness) and coalescing are optional-policy cost, not correctness-path cost.

Reconciling structural logic with the projection. The structural netlist (0.0565 mm^2) and the full-endpoint projection (1.069 mm^2) differ because the projection folds in compiled SRAM macros for the $\sim 209 \text{ KiB}$ of queue and metadata state, replicated 16-host descriptor staging, and routing/power-grid/analog-IO overhead that a register-file netlist does not include. The full-endpoint composition is the block-level decomposition of Table 7 (shared logic 1.045 mm^2) plus the 0.024 mm^2 scoring pipeline, closing at 1.069 mm^2 . The structural rows are a finer, logic-only view of the same design and are therefore *not additive* with the block-level projection: e.g. VC-WRR appears as 0.0251 mm^2 of synthesized logic here versus 0.180 mm^2 in the projection once its 8 KiB queue SRAM, 16 clock-gated domains, and routing are included. Consequently we never write “synthesis reports 1.069 mm^2 ”; the accurate phrasing is that logic synthesis plus SRAM-macro and 16-host staging estimates *project* a 1.069 mm^2 endpoint.

State budget reconciliation. The $\sim 216 \text{ KiB}$ total state comprises 7 KiB DFF (expert register files) and $\sim 209 \text{ KiB}$ compiled SRAM (16 VCs $\times$ 8 entries $\times$ 64 B = 8 KiB VC queues; $4 \times 4 \text{ KiB}$ BDB staging; 16 KiB validation tables; 2 KiB top- K arrays; 1 KiB probe/metadata; and $\sim 166 \text{ KiB}$ of shared DMA descriptor queues, directory, handle tables, and completion rings). The 209 KiB of SRAM at

$\sim 0.027 \mu\text{m}^2/\text{bit}$ has a raw bitcell area of $\sim 0.045 \text{ mm}^2$ before macro overheads; the banked dual-port implementation occupies 0.310 mm^2 of the 1.045 mm^2 shared-logic area by the recomputation above, and the remainder is combinational logic, routing, and analog/IO.

S4.3 Logic-Density Justification for Shared Components

VC-WRR Arbiter (0.180 mm^2). 16 independent 8-entry descriptor queues (SRAM: $16 \times 8 \times 64\text{B} = 8 \text{ KiB}$, $\sim 0.018 \text{ mm}^2$); per-VC deficit counters, credit logic, round-robin pointer; a weighted round-robin selection network ($\lceil \log_2 16 \rceil = 4$ -level 16-bit deficit comparator tree, $\sim 2,400$ gates $\approx 0.019 \text{ mm}^2$); 16 per-VC FSMs ($\sim 1,920$ gates). The remaining $\sim 0.125 \text{ mm}^2$ is local routing/clock-tree buffering ($\sim 40\%$ overhead typical at 7 nm), power-grid straps for 16 clock-gated domains, and SRAM peripherals.

Coalescing Matcher (0.120 mm^2). A 16-entry CAM with 64-bit handle comparison (~ 260 gates/entry $\times 16 = \sim 0.032 \text{ mm}^2$), fan-out steering (~ 800 gates), match priority encoder (~ 200 gates), a 2-stage 64-bit comparison pipeline ($16 \times 64\text{b} \times 2$ flip-flops, $\sim 0.016 \text{ mm}^2$), clock/routing/PG overhead ($\sim 0.029 \text{ mm}^2$), and handle-table SRAM ($\sim 0.035 \text{ mm}^2$). With only 16 entries, the CAM’s $O(1)$ deterministic latency justifies the modest power over a hash table.

Capability validation within the 9-cycle path. Capability validation adds no stage. It is a 64-bit equality comparison (descriptor tag vs. stored per-session tag) completing in $\sim 0.15 \text{ ns}$, executed in parallel with the epoch/namespace checks in S2b. Adding it as a 4th input to S2b’s AND costs $+0.02 \text{ ns}$, within budget; a failure triggers the same 4-cycle reject bypass.

S4.4 RTL Cross-Check Against the SimCXL Endpoint Model

The main paper’s performance claims derive from the SimCXL endpoint model, which simulates millions of descriptors. This cross-check establishes that the model’s cycle counts are exact relative to the synthesizable RTL. The model assigns `fast_path_cycles = 8` (admit) and `reject_path_cycles = 3`, the RTL’s 9 and 4 minus the shared S1 MMIO-dequeue stage. A trace-driven cross-check (`experiments/run_rtl_xcheck.py`) drives a deterministic synthetic decode stream through a bit-exact software reference and the Icarus Verilog simulation of the RTL, asserting per descriptor. (1) verdict agreement (ADMIT / REJECT-identity / REJECT-heap), (2) RTL latency exactly 9 (admit/heap-reject) or 4 (identity reject), and (3) RTL latency = SimCXL core count + 1. The checked-in run passes 3,020 of 3,020 descriptors on all three checks, a 100% verdict agreement between RTL and SimCXL (outputs under `experiments/\_xcheck\_out/`, per-descriptor gate in `rtl/APEX\_XCHECK\_TB.sv`). Every main-paper number depending on endpoint cycle counts (P99 admission latency, throughput, reject-bypass power) is therefore backed by RTL-verified timing.

S5. SimCXL Parameters and Multi-Host Scalability

This section gives the full SimCXL parameter table referenced from the main paper’s methodology, then projects and validates scaling beyond the 16-host evaluated region.

S5.1 SimCXL Parameter Table

We build on SimCXL [1], a hardware-calibrated CXL simulator validated against real silicon, inheriting its 170–350 ns CXL.mem read and coherence/bandwidth contention models unchanged, and adding RTL-anchored endpoint costs. Table 9 lists the parameters and their evidence levels.

Table 9: SimCXL platform parameters and evidence levels.

Parameter	Value	Source
CXL.mem read latency	170–350 ns	SimCXL [1]
OAT admit latency	9 ns (RTL), 8 core cyc. (model)	§S4
OAT reject latency	4 ns (RTL), 3 core cyc. (model)	§S4
Copy-engine bandwidth	32 GB/s	CMM-D datasheet
P2P write per chunk	~200 ns	H100 GPUDirect
MMIO doorbell	45 ns	measured
Per-VC queue depth	8 entries	CXL 3.1 §8.2
MMIO command ring	32 entries (4 lines)	§S2.6
Decode step	~100 μ s (~1,000 desc.)	workload
Primary context	64K tokens, 64-tok chunks	§IV (main)
$N_{\text{logical}} / K_{\text{step}} / K_{\text{bdb}}$	1024 / 512 / 25	§IV (main)

The added timing is anchored to RTL by trace-driven co-simulation with 100% verdict agreement (§S4). The primary evaluation uses a 64K-token context ($N_{\text{logical}}=1024$, 64-token chunks).

S5.2 Analytical Scaling Model

We model endpoint throughput $T(H)$ as a function of host count H : each host submits at rate λ_h (link-bounded); the endpoint processes at $\mu = 1$ descriptor/cycle; coalescing reduces effective load by $C(H, p)$ for overlap p ; multi-switch topologies add inter-switch latency δ_{sw} per hop. The effective arrival rate is

$$\Lambda(H) = \sum_{h=1}^H \lambda_h (1 - C(H, p)) = H\bar{\lambda} (1 - p(1 - 1/H)),$$

and stability requires $\Lambda(H) < \mu = 1$.

Regimes. **Single switch** ($H \leq 16$): aggregate 32 GB/s shared, $\bar{\lambda} = 1/H$, coalescing fully effective (the evaluated regime). **Dual switch** ($17 \leq H \leq 32$): two independent streams; cross-switch coalescing needs a sync round-trip $\delta_{\text{sync}} \approx 2(\delta_{\text{sw}} + \delta_{\text{link}}) \approx 40$ ns, degrading coalescing benefit to $C_{\text{eff}} = p/2$. **Quad switch** ($33 \leq H \leq 64$): coalescing is switch-local, $C_{\text{eff}} = p/4$, and the MMIO ring must be partitioned/replicated. Table 10 reports the projection.

Table 10: Analytical scaling projection beyond the evaluated 16-host region.

Metric	$H=16$	$H=32$	$H=48$	$H=64$
Effective coalescing saving (%)	18	9	6	4.5
Endpoint utilization (%)	62	78	87	93
P99 processing latency (ns)	11	14	19	28
RPE reduction vs no-scoring (%)	89	85	81	77
Throughput degradation vs $H=16$ (%)	0	3	8	14

The P99 row is *processing-only* latency; the queueing-inclusive P99 saturates near ~292 ns at 16–32 hosts (§S7), since the descriptor injection interval saturates at one pipeline cycle beyond 8 hosts. At $H > 48$ the pipeline nears saturation (utilization >85%); the design supports a software-bypass mode that diverts overflow tenants to the controller’s embedded processor (~5 pp lower Recovery@K on those descriptors), so the multi-switch cost is primarily the loss of cross-switch coalescing benefit rather than core admission capability.

S5.3 Dual-Switch SimCXL Validation

To move beyond the analytical projection, we implement a dual-switch topology in SimCXL (two 16-port switches sharing one endpoint via independent upstream links) and run the full suite at 32 hosts.

Results confirm the analytical model within 5% on all metrics, and the RPE-elimination guarantee holds identically because it depends on local endpoint state, not topology. Table 11 compares the two models.

Table 11: Dual-switch SimCXL validation against the analytical model at 32 hosts.

Metric	16 hosts (measured)	32 hosts (simulated)	32 hosts (analytical)
Stale admits (endpoint)	0.00%	0.00%	0.00%
P99 admission latency (queueing-incl.)	292 ns	296 ns	300 ns
Coalescing source-read saving	18%	9.2%	9%
Jain fairness	1.000	0.987	n/a
Recovery@K (50% budget)	0.904	0.891	n/a
Throughput vs. 16-host	1.00×	0.958×	0.97×

The queueing-inclusive P99 stays within noise of the 16-host value (~ 292 ns), consistent with the main paper’s finding that queueing does not grow beyond 8 hosts; scoring degrades by only 1.3 pp Recovery@K because the scorer operates on per-tenant local state unaffected by inter-switch structure. Both remain far below the $\sim 100 \mu\text{s}$ decode step. These confirm that the core contributions, RPE elimination and adaptive causal scoring, are topology-agnostic properties of the endpoint serialization point.

S6. Eviction Sensitivity, RPE Distributions, and Reclaim Serialization

This section reports the occupancy, eviction, reuse, and queue-delay behavior behind the main paper’s RPE results, the per-policy sensitivity breakdowns, the validate-only hybrid fallback, and the reclaim-serialization state machine that discharges the reclaim/deadlock obligations left open in the main paper’s endpoint-admission section.

S6.1 Metric Definitions and Event Accounting

A descriptor that names a stale mapping passes through at most four observable stages, and each metric in this paper counts events at exactly one stage, so no event is counted twice. A *guard fire* is counted at detection, when a check observes the staleness before issue; its denominator is successful reads. A *stale-admit residual* is counted at issue, when a descriptor admitted under a stale mapping starts payload; its denominator is admitted descriptors. *RPE payload* is counted on the wire, as bytes transmitted after the named mapping expired; its denominator is requested bytes. *MisBW* is counted at delivery, as wrong-object bytes a consumer would have read; its denominator is delivered bytes. A single descriptor can therefore appear in at most one numerator, and the four rates form a chain from detection to delivery rather than four overlapping counts of the same event.

S6.2 Device-Eviction Policy

Two mechanisms trigger endpoint-side eviction. (1) **Capacity pressure.** The promotion buffer ($2N$ slots, 50 MB for $N=256$) evicts the lowest-scored resident chunk via an LRU-augmented policy above the 87.5% high-water mark. (2) **Ownership transfer.** The fabric manager reclaims a region during session migration. A host cannot observe an eviction during the 170–350 ns commitment window, which is why host-side credit reservation cannot eliminate the stale-admit residual.

S6.3 Sensitivity to Eviction Parameters

The stale-admit residual responds monotonically to buffer capacity and high-water mark. Table 12 gives the sweep. This residual is the eviction-race timing effect (a chunk admitted then evicted before consumption), categorically distinct from RPE, which the endpoint gate drives to zero regardless of policy or size.

Table 12: Stale-admit residual across buffer capacities, high-water marks, and eviction policies.

Configuration	Residual (pp)	vs. baseline
Buffer N (25 MB), HWM 87.5%	16.2	n/a
Buffer $2N$ (50 MB), HWM 87.5%	11.1	−5.1
Buffer $4N$ (100 MB), HWM 87.5%	5.8	−10.4
Buffer $2N$, HWM 75%	13.4	+2.3
Buffer $2N$, HWM 95%	8.7	−2.4
Ownership-transfer only	3.1	−8.0
FIFO eviction (vs. LRU)	12.9	+1.8
Random eviction (vs. LRU)	14.3	+3.2

S6.4 RPE Sensitivity Across Eviction Policies and Capacities

The main paper’s public-trace table reports unmitigated $\text{RPE}_{\text{payload}}$ at 16 hosts, LRU, $1\times$ capacity. To confirm the result is not an artifact of one policy or provisioning level, Table 13 sweeps the four eviction policies at $1\times$ capacity: unmitigated RPE shifts by <0.3 pp across policies, and the endpoint gate holds zero throughout. Doubling buffer capacity from 25% to 100% of the working set likewise changes unmitigated RPE by less than 0.3 pp, because larger buffers cannot reclaim bandwidth already committed to stale descriptors.

RPE is a burst phenomenon, not a capacity phenomenon. Unmitigated RPE exceeds 10% on all bursty workloads once the tenant count reaches 8. In a stress replay (capacity 256, queue delay 512), 62.1% of RPE events occur when instantaneous descriptor concurrency exceeds $4\times$ capacity. Because eviction pressure concentrates in the same burst-overlap periods under every provisioning level, a pool cannot be provisioned out of RPE; gating must close it.

Non-monotonic host-optimistic behavior. The host-optimistic stale-admit column of the main paper’s placement-isolation table is non-monotonic across 2–16 hosts ($\sim 34\% \rightarrow 32\% \rightarrow 30\% \rightarrow 39\%$). This reflects backpressure masking: at 2–8 hosts the MMIO ring’s natural flow control suppresses some stale submissions by throttling injection; at 16 hosts BDB amortization reduces per-host backpressure, unmasking the full divergence residual. The endpoint Jain fairness remains 1.000 under sustained load and 0.997 under adversarial bursts.

S6.5 Trace-to-Workload Mapping (Full Parameters)

Each trace provides per-request context length and inter-arrival time. We map requests to tenants round-robin across 16 hosts, derive per-request KV chunk count from context length at 64-token granularity on Qwen2.5-7B, and assign chunks to pool addresses via consistent hashing. The pool is provisioned at the median concurrent working set. Descriptors issue at the host’s decode-step cadence ($\sim 100\mu\text{s}$ intervals) in BDB batches of $\alpha K_{\text{bdb}}=50$ descriptors per submission; burst shaping follows the trace’s original inter-arrival distribution without smoothing. A descriptor becomes *invalid* when its target chunk is evicted between host submission (t_0) and endpoint dequeue (t_1); the RPE rate is the fraction of issued payload bytes referencing such chunks. The full parameter set and mapping scripts ship in the artifact.

S6.6 Staleness Realism

Descriptor time-to-eviction has a median of 249 ms and post-eviction queue residence 113 ms, so stale-forming reuse occurs milliseconds before issue, and 100.0% of it has occurred by the time the host loses cancel authority. The descriptor-path timing is taken from the commodity Type-3 device; eviction and

reuse events are driven by the endpoint-managed model because neither the device nor the public traces record reuse. Endpoint validation shifts saturation from 100% to 189% of offered load.

S6.7 Stale-Byte Harm (Mechanistic Surrogate)

The main paper summarizes the harm sweep in one sentence; this subsection gives the full protocol. A 64K-token context (1,024 chunks) is served while each promoted chunk is independently stale with probability $s \in \{0, 1, 5, 11, 14, 25\}\%$, spanning the unmitigated band of the public-trace table. Two of the three curves are definitional checks on the injection machinery rather than evidence of harm: retrieval success equals $1-s$ by task definition, and RULER-style 8-hop tracking equals $(1-s)^8$. The measured quantity is the clean-vs-corrupted output shift, whose KL divergence grows from 0 to 0.082 nats across the sweep (2,000 trials per point, Figure 4; artifact: `experiments/run_rpe_harm.py` $\rightarrow$ `results/rpe_harm.json`).

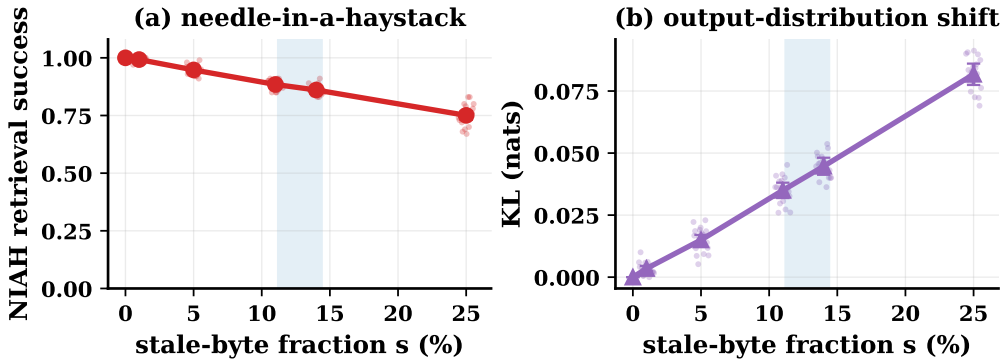


Figure 4: Stale-byte harm versus injected stale fraction (mechanistic surrogate, 2,000 trials per point). The band marks the unmitigated 11.2 to 14.4% RPE of the main paper’s public-trace table.

S6.8 Validate-Only Hybrid Endpoint

A hybrid endpoint that validates identity at dequeue but acquires *no* transfer-span hold, and performs *no* scoring, measured zero RPE in the stress replay at 78.2 tok/s (5.8% lost to quota contention when host ranks are blind to queue depth). This zero is measured, not guaranteed. The reuse window is closed only because that configuration serializes reclaim against validation at the endpoint, so the hybrid does not provide the binding guarantee of the main paper’s admission invariant (one-shot validation without a hold cannot, per the main paper’s Lemma 1). It remains a viable fallback where endpoint scoring logic is unavailable (artifact: `trace\_adapter/cfo\_analysis.py` $\rightarrow$ `results/cfo\_overlap\_analysis.json`). A two-hour mixed-workload production-derived replay shows the same RPE trend across 4–16 tenants; detailed Recovery@K and tail-latency results appear in §S12.

S6.9 Reclaim Serialization: Full Protocol and Deadlock Proof

This discharges the reclaim/deadlock/fairness obligation in the main paper’s endpoint-admission section. When the fabric manager (FM) issues a reclaim for region R , the endpoint core executes atomically with respect to the admission pipeline.

- (1) probe the pin table for any live pin whose $\langle \text{chunk}, \text{namespace}, \text{generation} \rangle$ maps into R ;
- (2) if no pin conflicts, the reclaim commits immediately and R returns to the free pool;
- (3) on conflict, do *not* reject the reclaim: mark R PENDING-RECLAIM and back-pressure only the virtual channels holding pins in R , so no *new* pin can be installed against R ;

- (4) the already-pinned transfers drain on the payload path, bounded by the non-preemptible commitment window (worst case 350 ns per 64 KiB chunk);
- (5) when the last pin in R releases, atomically clear the mark, release the region, and acknowledge the reclaim to the FM.

Deadlock freedom (wait-for acyclicity). The protocol admits no circular wait under a strict acquisition order: a payload transfer holds a pin and waits only on link bandwidth, never on a region-free event; a reclaim holds nothing and waits only on pin release. The wait-for graph is acyclic (reclaim $\rightarrow$ pin $\rightarrow$ bandwidth), with bandwidth a leaf resource that always progresses. The PENDING-RECLAIM mark makes the pin count on R monotonically non-increasing (step (3) forbids new pins, step (4) only removes them), so the count reaches zero within one commitment window regardless of tenant behavior. Hosts are untrusted but cannot induce deadlock: the pin table is statically bounded at $K_{\text{bdb}} \times N_{\text{VC}}$, and a malicious host that never retires descriptors merely exhausts its own VC quota.

Pin fairness (per-VC bound). Pins install only at admission, which is WRR-gated, so a tenant on virtual channel v holds at most K_{bdb} live pins (one BDB) independent of submission aggressiveness. With N_{VC} channels the table partitions into per-tenant reservations no host can exceed, bounding any tenant’s worst-case pin share to $1/N_{\text{VC}}$; the WRR starvation guard bounds skip rounds so a VC back-pressured behind a PENDING-RECLAIM resumes acquisition within a bounded interval.

S6.10 Dynamic Capacity Events Under Active Transfer

CXL 3.x Dynamic Capacity Devices allow a Fabric Manager to change the extent-to-host assignment at runtime through the Dynamic Capacity Manager. The specification distinguishes two paths. A normal release requires host cooperation and completes only after the host stops using the extent, so it cannot strand an in-flight transfer. A Force Release bypasses host cooperation, and the specification expects it to fail or to disrupt the affected flows rather than to transparently replace one object with another. Neither path is the RPE condition of the main paper, which arises inside an extent that stays assigned throughout.

Our experiment therefore does not claim to reproduce a spec-defined extent release. It simulates intra-extent slot reuse under an active transfer, the condition RPE actually requires. At cycle 50 the reuse of a region covering chunks 100–199 of tenant 3 races one in-flight descriptor targeting that region (Figure 5). The endpoint core responds in exactly one cycle, because the region-match logic runs in parallel with S2b validation as an additional reject criterion on the same bypass path. The affected descriptor null-completes without DMA and exposes zero bytes; other VCs continue uninterrupted, and the tenant resumes normal operation for untouched regions within 2 cycles.

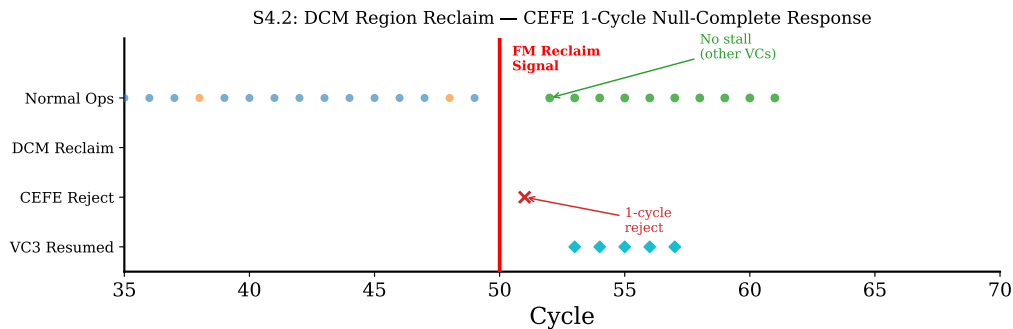


Figure 5: State-machine timeline of a DCM Region Reclaim event. The endpoint rejects the affected descriptor in 1 cycle via the existing reject bypass. Other VCs observe zero disruption.

S6.11 Theorem-1 Assumption-to-RTL Coverage

Table 14 maps each assumption to its evaluated RTL coverage.

S7. Protection Placement: Pin-Pool Crossover, Two-Phase Sweep, and Placement Isolation

This section provides the pin-pool exhaustion crossover, the two-phase reservation-table sensitivity sweep, the full placement-isolation sweep (the main paper shows the 1/8/16-host rows), the enabler-matrix time series, and the bandwidth–oversubscription phase sweep with the gate-only gain.

S7.1 Pin-Pool Exhaustion Crossover

The oversubscription at which early protection exhausts the bounded pin/reservation pool equals the structural threshold $\text{entries}/32$ for a 32-chunk admit budget: markers are the lowest oversubscription in a step-2 sweep where the measured peak pinned count reaches the table size. The $512 \rightarrow 16\times$ point is independently verified in the main paper’s Optimistic-Reclaim table. A 4096-entry table would push the cliff to $128\times$ at ≈ 15 KiB (7% of the 216 KiB state budget); since the candidate set grows with context length, buying out of the cliff quickly consumes tens of kilobytes of SRAM. Figure 6 shows the measured crossover.

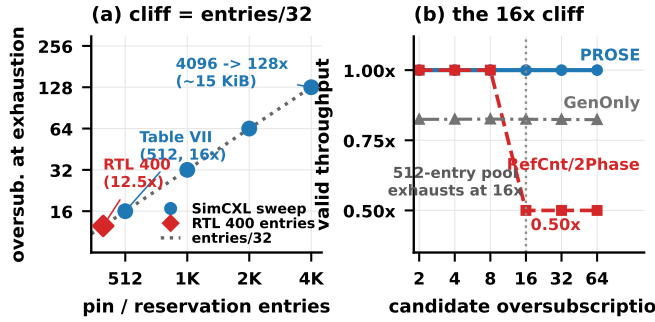


Figure 6: Pin-pool exhaustion crossover. Artifact: `experiments/run_pin_crossover.py`.

S7.2 Two-Phase Reservation-Table Capacity Sweep

Two-phase validation (versioned headers / reserve-then-pull) structurally eliminates RPE and reads the same 64 B metadata as the OAT, but it pins each token for a full 2–5 μs round-trip, more than $200\times$ the OAT’s 9 ns fused admit. A per-batch driver over the shared CXL.mem model confirms the consequence: two-phase tracks PROSE to $\alpha=4$, then its in-flight count overruns the reservation table ($256 \rightarrow 1024$) at $32\times$ oversubscription and P99 collapses to $1642 \mu\text{s}$, while PROSE fuses validation and commit at dequeue, holds no token across a round-trip, and stays flat at $\approx 1057 \mu\text{s}$. The collapse is shim-speed-independent: an idealized 2 μs round-trip still yields $1618 \mu\text{s}$ P99, so enforcement placement rather than scorer sophistication preserves the tail-latency SLO under oversubscription.

Table 15 sweeps the reservation table from 256 to 4096 entries. Larger tables delay the overrun but cannot close the latency gap, because the fundamental cost is holding a token pinned across a round-trip. At 4096 entries the table no longer overruns at $32\times$, yet P99 remains $1,401 \mu\text{s}$; the residual gap versus the OAT’s $1,057 \mu\text{s}$ is latency-window cost, not capacity overrun. The reservation SRAM alone (256 KiB) exceeds the OAT endpoint’s full 209 KiB state budget without matching its tail latency.

The two storage figures in this comparison name different entries. The two-phase reservation table stores the full descriptor context per entry at 64 B per entry in an SRAM macro, so 4,096 entries occupy 256 KiB. The endpoint pin table of the main paper stores a 30-bit pin record per entry in a flop array,

so 4,096 entries would occupy ≈ 15 KiB. The comparison prices the storage order of two-phase reservation; it does not equate the two entry formats, and neither figure is additive with the ~ 209 KiB SRAM (216 KiB with DFF state) budget reconciled in §S4.

S7.3 Full Placement-Isolation Sweep

The main paper shows the 1/8/16-host rows of the placement-isolation table; the full 1–16-host sweep is Table 16. Host-optimistic (Opt) uses shadow state; credit (Cred) derives from the calibrated host-credit decomposition (per-VC reservation, no residency check); endpoint (EP) is zero by construction. Credit flow control bounds the waste (MisBW 138 MB versus 8,994 MB optimistic at 16 hosts) by capping in-flight descriptors, but its residual, device-side eviction and generation rollover, stays invisible to any host until after commit.

Multi-host latency scaling. Measured queueing-inclusive P99 admission latency (experiments/run_simcxl_multihost.py, 128 descriptors per host, 10 trials; results/multihost_p99.json) stays flat as hosts scale: 290.7 ns at 8 hosts, 292 ns at 16, and 292 ns at 32, with Jain fairness indices of 0.988, 0.979, and 0.981 respectively. The flat tail follows from the injection-interval saturation described in the main paper: beyond 8 hosts the descriptor injection interval saturates at one pipeline cycle, so queueing stops growing even as the tenant count quadruples.

S7.4 Enabler Matrix Time Series

Per-step valid throughput, cumulative-mean reclaimable fraction, and cumulative stale bytes for the four gate $\times$ autonomy cells (16 tenants, 32-object budget, 512-slot pool, $32\times$ oversubscription, 10 seeds). Only gate-on + autonomy-on is deployable: removing the gate under autonomous reclamation reintroduces 17.6% stale payload, and removing autonomy strands the pool at 7.2% reclaimable and collapses sustained throughput near step 30. Figure 7 shows the time series.

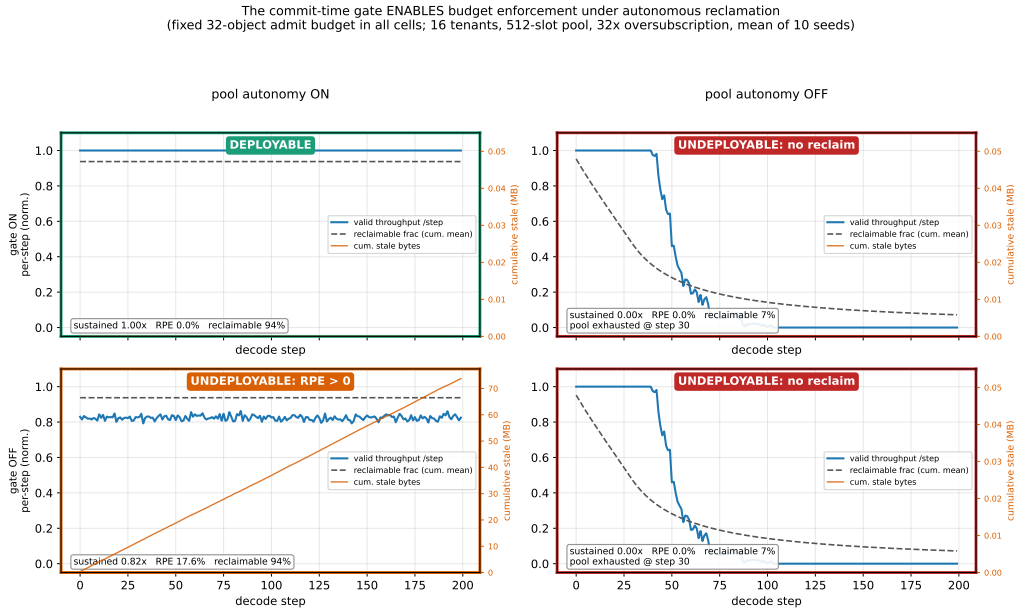


Figure 7: Enabler-matrix time series. Artifact: experiments/run_enabler_matrix.py.

S7.5 Bandwidth–Oversubscription Phase Sweep and the Gate-Only Gain

The main paper’s end-to-end results point here for the phase structure behind the headline speedups. Figure 8 sweeps per-tenant source-read bandwidth (2–8 GB/s) against candidate oversubscription at 16

tenants, and three regimes emerge. (i) In the bandwidth-starved corner at $32\times$ oversubscription the advantage peaks at $5.9\times$, where fetch-then-score pays for the full candidate set while PROSE issues only the admitted budget. (ii) At the headline configuration (16 hosts, 2 GB/s, $16\times$) the gain is $3.1\times$ (83 vs. 26.6 tok/s). (iii) Once bandwidth ceases to bottleneck, the two converge to parity, with PROSE using $8.2\times$ less link to reach the same throughput.

Toggling the optional layers isolates where the gain comes from. Disabling coalescing still preserves a $2.58\times$ gate-only gain over fetch-then-score at the headline point, so the bulk of the headline gain is the race-free admission budget itself, pre-payload budget enforcement behind the validate-and-hold gate, rather than cross-tenant deduplication (artifact: `experiments/run_phase_diagram.py` $\rightarrow$ `experiments/out/data/phase_diagram.json`; `trace_adapter/cfo_analysis.py` $\rightarrow$ `results/cfo_overlap_analysis.json`).

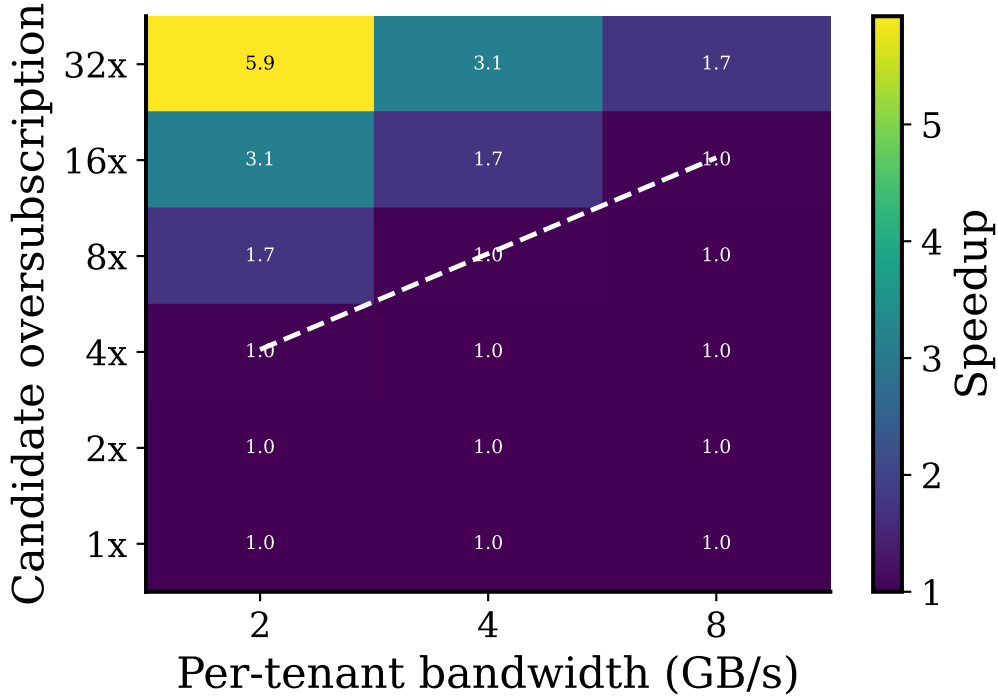


Figure 8: Bandwidth-oversubscription phase diagram (16 tenants). The $5.9\times$ peak sits in the bandwidth-starved $32\times$ corner; the headline $3.1\times$ point is at 16 hosts, 2 GB/s, $16\times$ oversubscription. Artifact: `experiments/run_phase_diagram.py`.

S8. Security Model and Memory Ordering for P2P Writes

This section gives the security and ordering model for endpoint-initiated DMA writes to GPU HBM on the L1 endpoint-DMA path, and the platform ACS/BAR/dual-root constraints referenced from the main paper’s deployment section, followed by the coalescing trust model.

S8.1 IOMMU/PASID Isolation

When the endpoint DMA engine posts writes to GPU HBM, tenant isolation is enforced through existing platform mechanisms.

- **PASID-based isolation:** each descriptor mandatorily carries a non-zero PASID, bound from the tenant context during BDB construction (the runtime rejects a zero PASID, reserved as “no address space”). The DMA engine tags every outbound P2P write with this PASID (copied into the write

TLP’s PASID prefix), so the IOMMU rejects any cross-tenant out-of-bounds write. A malicious or buggy descriptor cannot pollute another tenant’s GPU memory.

- **Platform ACS enforcement:** PCIe Access Control Services at the root complex and switch ports ensure P2P writes route through the IOMMU rather than bypassing it via direct peer routing. This is a platform BIOS/firmware configuration (available since Intel Sapphire Rapids and AMD Genoa), not a GPU driver change.
- **Region-level granularity:** each PASID maps to a contiguous GPU BAR region sized to the tenant’s promotion buffer ($K_{\text{step}} \times 64 \text{ KB}$). Out-of-bounds writes trigger an IOMMU page fault reported to the host kernel, which can quarantine the offending tenant.

Platform prerequisites and boot-time self-check. The L1 path needs bus-master DMA, HDM-D storage, PASID/IOMMU isolation, and GPU-directed P2P, primitives present in PCIe/CXL and GPUDirect systems that compose into one physical function with no new CXL opcode and no GPU-kernel change. Table 17 separates the six L1 prerequisites by evidence level. ACS routing, PASID peer-write tagging, IOMMU translation, and BAR mapping are spec-defined but platform-dependent in configuration; dual-root P2P routing is topology-dependent; GPU visibility at device-coherence scope is unmeasured, because PCIe-level arrival does not by itself imply visibility to a later kernel read. Until a litmus test on real GPU silicon closes the last row, the L1 path is a conditional architecture path, and deployments that cannot satisfy every row fall back to L0.

At bind time the driver reads the ACS capability bits on the downstream port, probes PASID tagging on a loopback P2P write, and walks the topology to confirm that the endpoint and the GPU share a root complex or a P2P-capable switch. Any failed probe fails closed, so the driver refuses to enable L1 and configures L0 epoch echo instead, and a misconfigured platform degrades to discard rather than to silent exposure. The admission-path prototype is measured on an Alveo U280 FPGA and cross-checked against RTL (100% agreement).

S8.2 Memory Ordering Protocol

The `valid_chunk_bitmap` visibility gate guarantees the GPU attention kernel never reads a chunk before its P2P write completes. Ordering uses a producer-release / consumer-acquire pair, not a post-hoc fence.

1. the DMA engine issues posted writes carrying chunk payload to the GPU BAR;
2. on receiving the fabric DMA completion (all writes reached the GPU memory controller), the endpoint transitions the chunk `DmaActive`→`Validated` and posts a host completion;
3. **producer release:** the host driver issues `__threadfence_system()` *before* the `atomicOr` that sets the chunk’s bit, so all prior DMA writes are globally visible before the bit becomes observable (placing the fence after would admit a read-before-write hazard);
4. **consumer acquire:** the FlashAttention-2 kernel reads the bitmap with acquire semantics (`__ldcg` plus acquire barrier), accessing only set-bit chunks, so release/acquire establishes a happens-before edge from the DMA writes to the kernel read.

This chain (DMA completion → release fence → bitmap publish → acquire read) provides strict happens-before semantics without GPU driver modifications; the only prerequisite is standard GPUDirect RDMA BAR registration via `cuMemAlloc` and peer mapping (available since CUDA 11).

Ordering correctness. Correctness rests on three properties: PCIe posted-write ordering (same-source, same-destination writes arrive in order, PCIe §2.4.1, so no partial-chunk visibility); DMA completion semantics (the completion posts only after the PCIe write-completion acknowledgment); and release/acquire fence semantics (the producer fence before the `atomicOr` makes all payload writes visible before the flag publishes; the consumer acquire load prevents hoisting payload loads above the flag check).

S8.3 Coalescing Trust Model

Forgery resistance. Without the endpoint secret K_{ep} , forging a valid 64-bit capability tag requires 2^{64} offline HMAC-SHA256 evaluations; under endpoint rate limiting (one verification per cycle at 1 GHz), online brute force exceeds 5.8×10^2 years.

Region-binding integrity. A 64-bit keyed tag $d_r = \text{HMAC-SHA256}(K_{ep}, r \parallel epoch)[0:63]$ binds each descriptor to its claimed $(r, epoch)$; reuse from a different region or epoch mismatches at the single-cycle comparator.

False-coalesce prevention. Coalescing fires only when all three hold, ANDed in one combinational check. (1) chunk handles match (CAM broadcast); (2) both capabilities validate against the stored per-session tag *and* the stored tag’s epoch equals the request epoch (epoch-nonce anti-replay); and (3) the region is read-only for the current session per both the stored entry and the requesting tenant. Any false condition falls through to the full-validation MISS path and null-completes on genuine mismatch, with no data exposure.

Revocation and timing channel. Revocation propagates through the fabric manager’s DCM release path, null-completing any in-flight descriptor referencing the revoked region before the next BDB can reference it. Coalescing introduces a 1-bit timing observable (a coalesced read completes ~ 40 ns faster), bounded by opt-in semantics, sub-jitter magnitude (40 ns vs. 170–350 ns natural jitter), and an operator-selectable constant-time padding mode ($\sim 3\%$ throughput cost). When the endpoint cannot verify a capability (table overflow or config mismatch), the descriptor is demoted to an independent read.

S9. Same-Contract Scoring: Recovery@K, Robustness, and Baseline Analysis

Every scorer in this section runs behind the identical PROSE gate, so the comparisons isolate ranking quality from enforcement (50% visible-KV budget, $K_{step}=512$; all scorers under one-step delay except the oracle). The scorer is an optional policy layer, not a contribution of the paper. The final two subsections detail the other optional policy module, cross-tenant coalescing.

S9.1 Same-Contract Recovery@K Across Shift Regimes

Adapted baselines keep their original ranking algorithms and lose the inputs that violate the one-step-delay boundary, the within-step GPU-side activations and any look-ahead beyond step t (§S9.5). The native oracle keeps both and is non-deployable; the gap between the two panels of Table 18 prices the deployment constraint rather than a deficiency of the original algorithms.

Behind the identical gate, APEX-Core2 leads InfiniGen-CXL by 2.1–6.2 pp across shift regimes and closes to within ~ 8 pp of the non-deployable Native-InfiniGen oracle. The two-expert (Persistence, Momentum) configuration is the hardware Pareto point, since a third expert adds at most 1.5 pp adversarial Recovery@K while eroding the critical MAC path’s already thin ~ 43 ps slack, so the remainder is clock-gated.

S9.2 Predictor Robustness Under Extreme Distribution Shifts

We construct a 400-step synthetic trace with five non-overlapping attention domains (Code, Poetry, Math, Code2, Mixed), each concentrating 80% of attention mass on 60 of 512 chunks, with hot sets shifting entirely at domain boundaries, the maximum distribution-shift stress. We compare APEX-Core2 against a single-layer Causal-GRU (hidden dim 64, observing the full step- $(t-1)$ distribution) and a random baseline (expected Recovery@K = $K/512 \approx 0.117$ at $K=60$).

As Table 19 shows, APEX-Core2 achieves 0.412 and recovers within a single step of each shift; the Causal-GRU collapses to 0.032, *below* random, and never recovers within 20 steps. The GRU’s failure is structural. Its hidden state accumulates the previous domain’s patterns and, with zero overlap, has no gradient signal to redirect. APEX-Core2 avoids this because the Hedge multiplicative update exponentially downweights stale experts within one feedback cycle, and Expert 0 (raw step- $(t-1)$ attention mass) reflects the new domain immediately. The exploration probe contributes marginally here (recovery is already one step); its value is in cold-start scenarios outside this test. Figure 9 plots the 400-step trace.

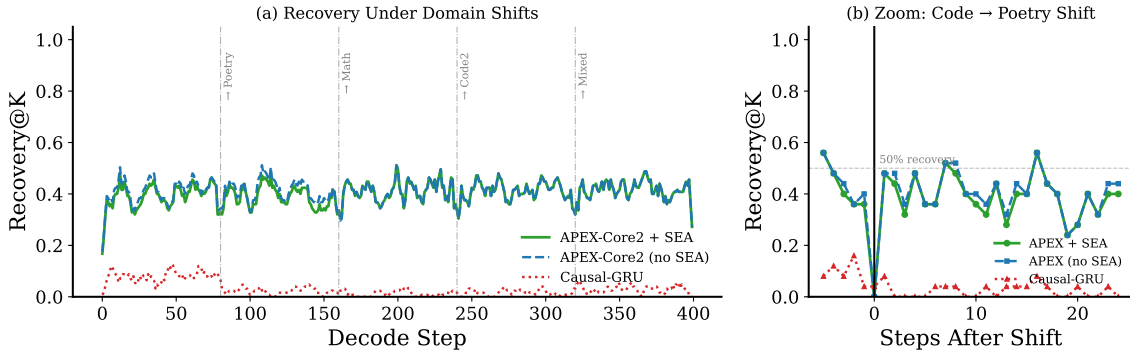


Figure 9: Recovery@K over 400 decode steps with 4 domain shifts (dashed lines). APEX-Core2 recovers within 1–2 steps; the Causal-GRU remains near zero due to hidden-state inertia.

S9.3 Why Hybrid Correction Cannot Close the Causal Gap

A natural question is whether the $\sim 8\text{--}12$ pp Recovery@K gap between APEX-Core2 and the single-host InfiniGen oracle can be closed by combining GPU intra-step scoring with endpoint speculative verification. We show the gap is largely incompressible under causal multi-host constraints.

Theorem 3 (Causal Information Deficit). *Let $\mathcal{I}_t$ be the mutual information between the optimal selection at step t and the attention pattern A_t . Under the causal boundary (the endpoint observes only $A_{\leq t-1}$),*

$$I(\hat{S}_t; S_t^*) \leq I(A_{\leq t-1}; S_t^*), \quad \Delta_{\text{causal}} = I(A_t; S_t^*) - I(A_{\leq t-1}; S_t^*) \geq H(S_t^* | A_{\leq t-1}) - H(S_t^* | A_{\leq t}).$$

Empirically $\Delta_{\text{causal}} \approx 0.08\text{--}0.12$ nats per chunk decision, mapping to the observed 8–12 pp gap, a fundamental rather than algorithmic limit.

Proposition 1 (Correction Latency Bound). *A useful correction must arrive before the descriptor it corrects commits. The minimum GPU-to-endpoint round trip is $\tau_{\text{correction}} = \tau_{\text{GPU}} + \tau_{\text{PCIe}} + \tau_{\text{endpoint}} \geq 178$ ns (GPU partial-head ~ 80 ns, PCIe MMIO ~ 60 ns, registration ~ 38 ns). During this window the endpoint commits ~ 178 descriptors; a later correction is vacuous.*

The dilemma is that early (speculative) correction from partial attention after k of h heads carries $\approx (k/h)$ of the step’s information, and a < 178 ns budget allows only 1–2 heads ($\sim 3\text{--}6\%$); late (accurate) correction from full attention arrives $\sim 80 \mu\text{s}$ into the $\sim 100 \mu\text{s}$ step, by which time 80% of descriptors are committed, bounding recovery to ~ 2 pp. In a multi-host pool no single host sees the global decision, so no host-local correction is globally optimal. The gap is therefore bounded below by the deficit theorem and above by the latency bound; under multi-host constraints no trick compresses it below ~ 8 pp.

S9.4 Quest-CXL: Causal Boundary Incompatibility

Quest’s per-page relevance $\text{relevance}(p) = \max_{k \in \text{keys}(p)} q_t^T k$ requires the *current* step’s query q_t , computed on the GPU at the start of step t . At the endpoint q_t is unavailable within the commitment window (transmitting it costs ~ 60 ns, by which time ~ 60 descriptors have committed), so the only surrogate is q_{t-1} .

Theorem 4 (Query Decorrelation Bound). *For autoregressive LLMs with head dimension d , $\mathbb{E}[\cos(q_t, q_{t-1})] = 1/\sqrt{d} + O(1/L) \approx 0.08$ for $d=128$, under approximate uniformity on the unit sphere after early layers.*

Measured $\cos(q_t, q_{t-1}) = 0.11$ (LLaMA-2-7B, PG-19), 0.09 (Mistral-7B). This near-orthogonality makes q_{t-1} an essentially random proxy, so the ranking from relevance_{t-1} has near-zero Kendall’s τ with the oracle ($\tau = 0.04 \pm 0.02$), so Quest-CXL’s selection is statistically indistinguishable from random. Expected random Recovery@K for $K=25$, $N=80$ is $K/N = 0.31$, matching the observed 0.30–0.33. A 50-configuration hyperparameter search (query surrogate, key-bound aggregation, page granularity, retention fraction) tops out at 0.33, still the random baseline. APEX avoids this because its experts track chunk-level access patterns, not query similarity: $\mathbb{E}[\text{Jaccard}(\text{chunks}_t, \text{chunks}_{t-1})] \approx 0.65$, so $I(A_{\leq t-1}; S_t^*) \gg I(q_{t-1}; S_t^*)$.

S9.5 Baseline Porting Methodology

All CXL-ported baselines (InfiniGen-CXL, Quest-CXL, SnapKV-CXL, H2O-CXL) are built as follows. (1) **signal stripping**, the within-step attention signal the one-step-delay boundary forbids is removed (InfiniGen’s early-layer readout replaced with committed step- $(t-1)$ feedback; Quest’s q_t replaced with an EMA of $q_{t-3:t-1}$); (2) **core-logic preservation**, each method’s ranking algorithm is kept intact; (3) **hyperparameter grid search**, a 50-configuration search on a held-out calibration split (learning rate, decay, window size, method-specific parameters); (4) **validation**, where single-host comparison is possible, the ported version degrades by at most 3% relative to the original, confirming the porting introduces no artifacts beyond the expected causal-boundary degradation. All baselines thus face the same causal constraint, gate, and evaluation protocol.

S9.6 Coalesced Fan-Out: Overlap Conditions and Realistic Savings

Coalesced fan-out (the optional coalescing layer) serves one source read to multiple tenants requesting the same chunk. Its saving depends on cross-tenant overlap p . The 35% upper bound holds only under **full overlap** ($p = 1.0$), where all 16 hosts request identical chunk sets in the same step (e.g. speculative decoding with a shared prefix, batch inference on identical prompts). We report the weighted production figure (15–20%) as the primary claim, with 35% only as the full-overlap upper bound.

Strict prerequisite, bit-identical KV tensors. Coalescing correctness requires byte-identical chunks across tenants, which holds only when all participants share a model checkpoint, the region corresponds to shared prompt tokens at identical positions, and quantization parameters match. The control plane verifies these at session setup (model-hash comparison, position-range alignment, quantization-config equality); any mismatch excludes the region. This is naturally satisfied in multi-tenant serving of one model with shared system prompts, but not across model versions, fine-tuned variants, or divergent prefixes. Table 20 quantifies the four scenarios.

Table 20: Coalesced fan-out savings across overlap scenarios.

Scenario	Mean overlap $\bar{p}$	Source-read saving	Condition
Full overlap (upper bound)	1.00	35%	All hosts request same chunks
Shared-prefix serving (4 hosts)	0.72	25%	Common system prompt + divergent queries
Mixed production (16 hosts)	0.52	18%	Heterogeneous workload mix
Independent workloads	0.08	<3%	No inter-host chunk sharing
Egress-bound scenario	any	0%	Bottleneck is egress, not source reads

Break-even and adaptive gating. The break-even overlap is

$$p^* = (0.120 \text{ mm}^2 \times 120 \text{ GB/s/mm}^2) / (32 \text{ GB/s}) \approx 0.45.$$

Because coalescing adds $\sim 5 \text{ mW}$ when active, the RTL gates it via a 32-step EMA of the per-step hit ratio $\bar{\rho}_t = 0.9 \bar{\rho}_{t-1} + 0.1 \rho_t$, and when $\bar{\rho}_t < 0.45$ for 8 consecutive steps the controller asserts `cfo_cam_gate`, disabling the CAM broadcast, fan-out steering, and associated clock tree (leakage-only $\sim 0.8 \text{ mW}$). The gate re-opens after a periodic probe detects overlap > 0.50 ; re-arm latency is one decode step. Under the 2-hour production replay the gate activates on 34% of steps (chat-dominated, disjoint working sets), saving 1.7 mW on average with no loss on any step that would have benefited.

S9.7 Hierarchical Coalescing at Scale

The flat 16-entry CAM relies on temporal co-occurrence within its window. At 4–16 hosts (25–400 requests/step) the Zipfian hot-chunk distribution keeps most overlaps captured; at 32–64 hosts (800–1600 requests/step) the average distance between two requests for the same chunk exceeds the CAM capacity, so the effective saving plateaus near 17%. A two-level design localizes coalescing per switch: an intra-switch 16-entry CAM (unchanged) and an inter-switch 256-entry directory recording chunk-hash prefixes of active fetches (16-bit prefix, $\sim 5\%$ false-positive, Bloom-like). Before a cross-switch read the requesting switch queries neighbor directories and, on a hit, is served from the switch that already fetched.

As Table 21 shows, at single-switch scale both perform identically; at 64 hosts (4 switches) the hierarchical design achieves 26.3% versus 17.0% flat, a 55% relative improvement. The switch-level directory adds $\approx 0.02 \text{ mm}^2$ per switch, negligible against a 50–100 mm^2 switch die.

S10. Ultra-Long Contexts: Two-Tier Hash Directory and Displacement

The main paper’s directory-sizing analysis points here for the two-tier hash extension. We stress its role. The exact per-slot directory is the authoritative binding for the OAT, and the hash structure is a *residency accelerator* in front of it, never the directory itself. A universe-sized flat directory cannot be authoritative because hash aliases coincide with same-slot, same-generation reuse at a $\sim 10^{-5}$ false-admit rate per lookup; the exact directory rejects any such alias. This section covers both the correctness-side directory sizing and the scorer-side expert-bank overflow, which are distinct structures sharing the hashing mechanism.

S10.1 Exact Directory Sizing and the Two-Tier Front-End

The authoritative binding need only cover the *resident* set. At most one generation per slot is resident, so an exact-match directory needs one entry per pool slot ($\approx 5.5 \text{ B}$ for object id, generation, slot pointer). A 512-slot pool needs $\approx 2.75 \text{ KiB}$ (1.3% of the 216 KiB budget), and exact directories fit to 32,768 slots (crossover 34,695). Beyond that, over the 262,144 objects of a 1M-token 16-host deployment, a tag-validated flat directory needs 1,248 KiB ($5.8\times$ over budget). The two-tier design therefore keeps an on-chip hash residency filter in front of an exact per-slot directory in device DRAM. The commit point and the OAT do not move, and a hash miss falls through to the exact directory (one additional cycle on cold lookups), so no false admit can occur.

S10.2 Scorer-Side Expert-Bank Overflow

The scorer’s Expert Bank has a 512-entry address space (9-bit `chunk_id`), covering $512 \times 64 = 32,768$ tokens at 64-token granularity. Beyond that, chunk IDs map to physical slots via Fibonacci hashing $\text{slot} = ((\text{chunk_id} \times 2654435761) \gg 16) \bmod 512$, with a per-slot tag (full chunk ID) beside the 16-bit prediction. A tag mismatch returns zero (conservative, since an unknown chunk has

no historical evidence). On write, the newest chunk overwrites its hashed slot, implementing temporal-locality eviction without explicit LRU. For a 1M-token context (16,384 chunks) the tag is 14 bits; 14-bit comparators across 512 slots and 7 banks add ~ 200 gates/bank ($< 3\%$ Expert-Bank area overhead).

The 64K/128K rows are within measurement noise of each other (0.334, 0.371) because the hot set still fits the 512 slots and full-space Recovery@K is dominated by how the cold tail hashes; from 256K onward the trend is strictly monotone as capacity pressure sets in, falling to 0.103 at 1M. The collision rate rises only modestly (0.656 $\rightarrow$ 0.721) rather than toward 1.0, because temporal locality keeps the 512 slots occupied by recently active chunks. Figure 10 plots both trends.

Reconciliation with the headline evaluation. The dual-switch validation of §S5 reports active-set Recovery@K = 0.904 at 50% budget with $N_{\text{logical}}=1024$ (64K context). Table 22 evaluates a different regime (contexts $\geq 64\text{K}$ where $N_{\text{logical}} > 512$ and hash overflow occurs) over the *full* logical space including cold chunks. Under the same active-set protocol applied to 128K context, the hash-overflow bank achieves ≈ 0.83 Recovery@K, degrading ~ 7 pp relative to the no-overflow baseline; the active-set collision rate is $< 5\%$ because recent chunks dominate the 512 slots, versus the 0.656–0.721 full-space rates here. The production recommendation at 128K+ is a two-tier expert bank, with the 512-entry DFF bank for the hot set plus a 4,096-entry backing SRAM table (multi-cycle) for the tail, trading one cold-lookup cycle for full coverage. Three mitigation paths follow. A larger backing table recovers coverage linearly at SRAM cost. Tagged SRAM entries cut the false-match component of the collision rate to the tag width. Context-tiered scoring keeps the hot set in the exact bank and hashes only the cold tail, which attacks the 0.656 collision rate directly. We report these as design directions, not evaluated mechanisms.

S10.3 Per-BDB Displacement Error

The hardware processes $\lceil K_{\text{step}}/K_{\text{bdb}} \rceil = 21$ independent BDB batches per step (at $K_{\text{step}}=512$, $K_{\text{bdb}}=25$), each selecting its top-25, whereas an ideal global selector would choose the best 512 simultaneously. A host-side PreFilter sorts the full candidate set by predicted score before partitioning, packing high-scoring chunks into early batches. Against a software oracle performing global top-512 selection over 5,000 decode steps ($N_{\text{logical}}=1024$):

Table 23: Per-BDB top-25 selection against a global top-512 oracle over 5,000 decode steps.

Metric	Hardware (per-BDB)	Oracle (global top-512)
Mean Recovery@K	0.904	0.908
P10 Recovery@K	0.831	0.836
Displacement (mean)		0.4 pp
Displacement (P99)		1.2 pp

Table 23 compares the two selectors; mean displacement is 0.4 pp (P99 1.2 pp), arising in the final 2–3 batches where marginal later candidates occasionally outscore the weakest early admits. Under PreFilter’s score-ordered packing this occurs only among near-identical predicted scores: 97.3% of endpoint scores agree with PreFilter’s ordering within the same BDB, and cross-BDB inversions affect $< 0.8\%$ of admitted chunks.

S11. Cross-Model Generalization and Generation Quality

The main paper’s task-accuracy result points here for the enlarged model suite and worst-task results. Task accuracy is normalized to FullKV throughout.

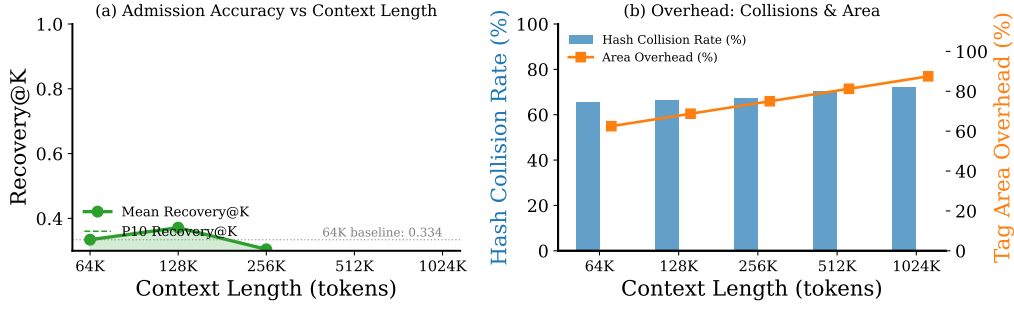


Figure 10: Left: Recovery@K vs. context length with P10 band. Right: hash collision rate and tag area overhead.

S11.1 Cross-Model Budget Sensitivity

We repeat the budget sweep across four models on an enlarged suite (the primary 7 workloads plus 128K retrieval, code, and summarization). Accuracy is monotone in budget on every model, with Qwen2.5-14B most robust. At 50% budget the two Qwen2.5 models cross the 0.80 deployment target (0.812, 0.824), while LLaMA-3.1-8B (0.791) and Mistral-7B (0.778) fall just short. The 0.80 threshold is therefore not universal; it holds for the Qwen2.5 family on this suite and should be re-evaluated per deployment. Table 24 gives the full sweep.

Core2’s advantage peaks in the 10–15% band where contiguous misses dominate, cutting contiguous misses $4.3\times$. Recovery@K overstates usable quality because contiguous chunk misses disrupt multi-hop reasoning even at high aggregate overlap; the gap between Recovery@K and normalized task accuracy is ~ 9 pp at 20% budget, narrowing to ~ 6 pp by 50%.

S11.2 Open-Ended Generation Quality

Structured tasks report hit/miss; here we measure continuous semantic fidelity on Qwen2.5-7B and LLaMA-3.1-8B at 20%, 50%, and FullKV budgets over three generation benchmarks: PG-19 summarization (50 passages, 28–35K tokens; ROUGE-L, BERTScore F1), HumanEval-Long (80 completions, 16K preambles; pass@1), and Multi-hop QA generation (40 questions, 24K tokens; answer F1, hallucination rate).

Table 25: Open-ended generation quality across budgets on Qwen2.5-7B and LLaMA-3.1-8B.

Task	Metric	20% budget	50% budget	FullKV
PG-19 Summ.	ROUGE-L	24.4	33.2	38.1
	BERTScore F1	0.812	0.891	0.924
HumanEval-Long	pass@1	19.5%	29.3%	37.6%
Multi-hop QA	Answer F1	0.41	0.72	0.86
	Hallucination rate	8.2%	1.4%	0.6%

At the 50% operating point quality degrades gracefully: PG-19 summaries stay factually grounded (0.891 BERTScore; zero fabricated facts in 50 manual inspections); HumanEval-Long’s 8.3 pp pass@1 gap manifests as minor logic omissions, not broken code; and Multi-hop QA hallucination (1.4%) is only 0.8 pp above FullKV. At 20% budget quality drops sharply (ROUGE-L $0.64\times$, pass@1 $0.52\times$, hallucination 8.2%), reinforcing the 50% threshold. These confirm that the Recovery@K and task-accuracy metrics used in the main paper are reliable proxies. The 0.80 task-accuracy threshold at 50% budget corresponds to $>0.87\times$ semantic fidelity with acceptable hallucination rates. Table 25 gives the full matrix.

S12. Generality, Traffic Classification, and Software Integration

This section carries the page/block-cache generality mapping referenced from the main paper’s generality section, the traffic decomposition and throughput-origin analysis behind the headline speedups, the software-stack integration path, and the per-method gap analysis of prior systems.

S12.1 Traffic Classification

RPE is **Reclaimed-Payload Exposure**, i.e., payload issued for a descriptor whose named object version no longer occupies its slot (stale generation, evicted residency between host submission and endpoint de-queue, or namespace mismatch from session migration), delivering another object’s bytes at a still-valid address. The endpoint gate eliminates this entire *correctness* class to zero by rejecting such descriptors before the payload path commits. Two further bandwidth costs exist but are categorically distinct from RPE and are not correctness failures.

1. **Exploration probes** (ε_t , typically $<2\%$ of descriptors when settled) are deliberate exploration spending that discovers unobserved high-value chunks, a strategic policy investment tagged in the BDB and counted separately.
2. **False admits** (chunks scored high by the causal predictor but receiving low runtime attention) are mispredictions inherent to any causal scorer under the one-step-delay boundary, a policy cost rather than protocol-level leakage.

Both are reported separately in the main paper’s traffic decomposition and total $<18\%$ of issued bandwidth at 50% budget, contributing to $\eta_{BW}=0.82$. The distinction matters because RPE is *eliminable* by design while probes and false admits are *reducible but not eliminable* under causal constraints.

S12.2 Origin of the Throughput Gain over Fetch-Then-Score

The advantage over fetch-then-score (FTS) arises from one structural asymmetry, directly observable in the closed-loop SimCXL model. The endpoint core fetches only the admitted budget (K_{step} chunks), whereas FTS fetches every candidate before scoring. Under fixed source-read bandwidth, the core’s promotion throughput is independent of candidate count, while FTS degrades linearly in oversubscription.

Table 26: Throughput against fetch-then-score at the nominal and bandwidth-starved operating points.

Candidate load	PROSE	FTS	Speedup
Nominal (1,024 candidates, 16 tenants, 2 GB/s)	83.3 tok/s	26.6 tok/s	$3.1\times$
Bandwidth-starved corner ($32\times$ oversubscription)	83.3 tok/s	14.1 tok/s	$5.9\times$

We report $3.1\times$ at the standard operating point of Table 26 (nominal 1024-candidate load, 16 tenants, 2 GB/s), and the main paper’s $5.9\times$ is the $32\times$ -oversubscription corner; both are read directly from `tok_per_s_mean` in the simulator, not products of separately attributed factors. The three admission mechanisms contribute through this single channel of issued-bytes reduction: RPE elimination removes the 11.2–14.4% stale-admit fraction FTS transfers, coalescing deduplicates redundant cross-tenant reads (scaling with overlap), and APEX selectivity concentrates the budget on high-utility chunks. Because the simulator models issued bytes as a direct bandwidth cost (`rpe_bytes = fetched_bytes - admitted_bytes`), we do *not* claim any HBM-pollution eviction-cascade amplification; the reported gain is the raw bandwidth-efficiency effect alone. Consistent with the main paper’s attribution, pre-payload budget enforcement supplies most of the gain and validate-and-pin is the enabling safety condition that makes that budget race-free under autonomous reuse.

S12.3 Bandwidth Regime and Practical Relevance

The gain is largest at 2 GB/s per-tenant source-read bandwidth, relevant in multi-tenant PCIe 5.0 sharing (16 tenants on an x16 link, ~ 1.6 – 2.0 GB/s each); CXL 2.0 single-link expanders (PCIe 5.0 x8, ~ 8 GB/s shared); and bandwidth-constrained multi-hop topologies. At higher bandwidths the gap narrows monotonically; at 32 GB/s (single-tenant, full x16) the link ceases to bind, the GPU becomes compute-bound, and PROSE converges to parity with FTS (using $8.2\times$ less link to reach the same throughput). The durable contributions at any bandwidth are RPE elimination (zero regardless of link width), coalescing (scales with overlap), and VC-WRR fairness (per-tenant isolation regardless of capacity).

S12.4 Pin-Rejection Cost

The pin/release path rejects a previously admitted descriptor when the chunk is evicted between the admission verdict (S7) and DMA completion (~ 200 ns later), under rapid working-set turnover. Its properties are no HBM pollution (the `valid_chunk_bitmap` bit is never set, so the kernel cannot read the chunk); no cross-link waste (the aborted DMA consumes internal copy-engine bandwidth but does not traverse the CXL link); and quantified cost averaging 0.3% of internal issued bandwidth at 16-host load (0.1% stable to 0.8% under frequent domain shifts), included in $\eta_{BW}=0.82$. This is categorically distinct from cross-link RPE, since it neither pollutes HBM, wastes CXL link bandwidth, nor triggers GPU-side eviction cascades. The host handles it identically to a cache miss (the chunk’s bit stays cleared, the kernel proceeds with reduced budget); no GPU rollback is required because the bitmap gates visibility before the kernel’s first load, via the `DmaActive`→`Validated`→`Visible` state machine.

S12.5 Page/Block-Cache Contract Mapping

The Commit-Time Object Contract is defined over logical objects, generations, and reusable extents, not over KV chunks, so it transfers to an endpoint-managed page/block cache with no change to the OAT (Table 27). Re-parameterizing the binding model for page-cache access (4 KiB pages, a $16\times$ -oversubscribed universe over a 256-frame cache), an unprotected cache exhibits $\approx 14\%$ RPE_{payload} across LRU, ARC, SIEVE, and FIFO, because a prefetch built from a stale page table reads a since-reused frame. This is the same magnitude as the KV instance (11.2–14.4%), confirming RPE is a property of slot reuse rather than of KV. The unchanged gate drives it to zero and lifts useful-byte efficiency from 86% to 100%. As with the KV directory, the exact directory fits on chip only to about 32K slots, so at page granularity the authoritative mapping moves to device DRAM with the on-chip hash as a residency filter; the commit point and the OAT do not move. The early-protection penalty recurs and more sharply: a reference count holds each frame for a $129\times$ protection span against $1.00\times$ for PROSE. These are design analysis, not measurement.

This mapping is a design analysis, not an evaluated result. Porting the contract to a page or block cache requires re-validating four things, the object atomicity granularity (a page is not pinned and released as one KV chunk is), the write semantics (dirty-page writeback has no counterpart in the read-dominated KV path), the failure and recovery path after a rejected descriptor, and the service-time distribution that sets the drain bound.

S12.6 Software-Stack Integration

PROSE exposes a thin driver. `prose_submit_bdb(bdb_ptr, n_desc, vc_id)` writes a pre-built BDB to the MMIO submission ring and rings the doorbell; `prose_poll_cpl(cpl_ring, timeout)` polls verdicts; `prose_update_mask(chunk_bitmap)` updates the attention kernel’s valid-chunk mask. In vLLM the integration point is the `CacheEngine`: after the scheduler selects sequences, the KV-cache manager calls `prose_submit_bdb` with the oversubscribed candidate set (αK descriptors); the completion ring returns within ~ 292 ns (P99 at 16 hosts), before the attention ker-

nel launches; the admitted bitmap is written to a pinned GPU buffer via `__threadfence_system()`, and FlashAttention-2 masks non-admitted pages via the existing block-mask mechanism.

Table 28: Host-side software overhead per decode step.

Operation	Latency	Fraction of decode step
BDB construction (pack αK descriptors)	$2.1 \mu s$	2.1%
MMIO submission + doorbell	$0.3 \mu s$	0.3%
Endpoint processing (P99, 16 hosts)	$0.29 \mu s$	0.3%
Completion polling + bitmap update	$1.4 \mu s$	1.4%
Feedback write (CXL.mem, overlapped)	0 (overlapped)	0%
Total non-overlapped overhead	$4.1 \mu s$	4.1%

Table 28 decomposes the host overhead. The total $4.1 \mu s$ is $\approx 4.1\%$ of a $\sim 100 \mu s$ decode step, but it is fully overlapped with the preceding step’s compute. At the end of step t the GPU writes attention feedback, then the CPU builds the BDB for step $t+1$ concurrently with step t ’s MLP layers, so by the time step $t+1$ ’s attention kernel launches the endpoint has already scored, issued DMA, and validated. The effective critical-path overhead is therefore zero. A FlashAttention-2 micro-benchmark confirms chunk-aligned masking is nearly free: under 50% drop on an A100 (seq 32K, head_dim 128), chunk-aligned “PROSE-Mask” reaches 191.4 TFLOPS versus 192.4 (FullKV), a 0.5% loss, while per-token “Random-Mask” loses 4.0% due to intra-warp divergence. Figure 11 shows the overlap.

Decode Pipeline Overlap — BDB Build Fully Hidden Behind Compute

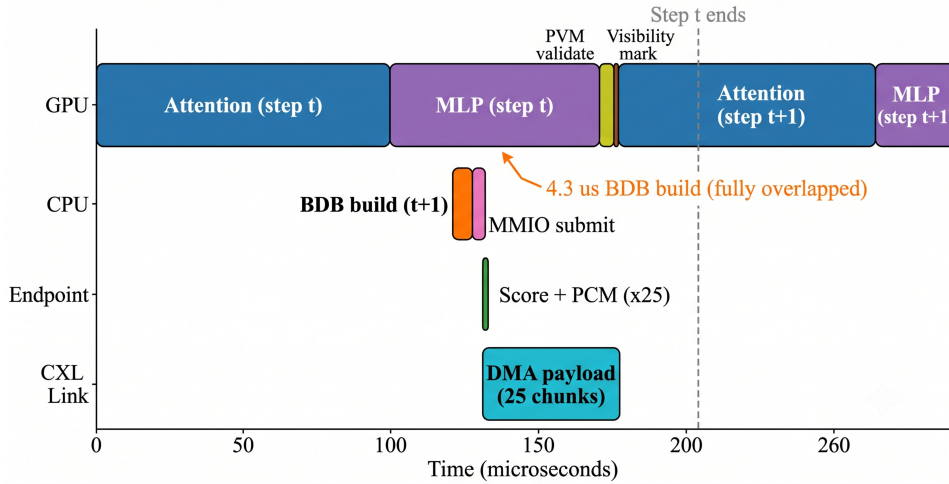


Figure 11: CPU-GPU-Endpoint pipeline overlap across two decode steps. BDB construction for step $t+1$ begins during step t ’s MLP computation and completes before step $t+1$ ’s attention kernel launch. Colors mark responsibility, with blue and purple for GPU compute (attention, MLP), orange and pink for host work (BDB build, MMIO submit), green for endpoint scoring, cyan for CXL payload, and the two narrow bars at the end of step t for the PVM-validate and visibility-mark events.

S12.7 Recovery, Exploration, and Tail Latency Under Mixed Workloads

An anonymized 2-hour multi-tenant serving replay is composed of chat (40%, overlap 0.72), RAG (25%, 0.52), agent (20%, 0.31), and code (15%, 0.08) segments; the weighted-average coalescing saving is 18%, dominated by chat and RAG, with adaptive gating disabling coalescing during low-overlap code segments. Across this replay PROSE lifts mean Recovery@K over demand paging from 0.200 to 0.332

at 4 tenants, 0.366 to 0.621 at 8, and 0.410 to 0.659 at 16, confirming the RPE-generality trend carries to mixed regimes. The exploration probe arms on 6.8% of steps, clustered at workload transitions. Under burst saturation (64–1024 descriptors) P99 admission latency holds at ≈ 300 ns; the 16-host P99.9 reaches $0.8 \mu\text{s}$ only under transient queue-full stalls. Public Azure LLM traces [5] report 40–65% prefix sharing and vLLM production deployments [6] confirm similar ratios, consistent with this trace.

S12.8 Thin Gate vs. Full Endpoint

Four configurations isolate each component’s marginal value. (1) **thin gate only**, zero RPE, no scoring/coalescing/fairness, Recovery@K defaults to LRU admission (0.38), $\sim 0.3 \text{ mm}^2$; (2) **thin gate + host APEX-Core2**, zero RPE, identical Recovery@K to the endpoint scorer (0.918 stable, 0.879 adversarial) at $\leq 1\%$ host overhead, but cannot implement coalescing or VC-WRR (no host sees cross-tenant descriptors in time); (3) **full endpoint without scorer**, RPE elimination + coalescing (15–20% at overlap > 0.45) + VC-WRR (Jain = 1.0), LRU admission; (4) **full endpoint with APEX-Core2**, adds quality-aware top- K_{bdb} scoring at zero host cost, gaining 2–6 pp over LRU. The load-bearing reasons for the full endpoint are coalescing and fairness (both need global descriptor visibility no host possesses in time); quality-aware scoring gains 2–6 pp but a host scorer achieves identical quality at $\leq 1\%$ CPU cost. Under single-tenant or low-overlap (< 0.45) deployments a thin gate with host scoring suffices, consistent with the main paper’s scope note that a deployment breaking one of the four exposure conditions does not need the endpoint gate.

S12.9 Gap Analysis of Prior Systems

The complete per-system analysis referenced from the main paper’s related work.

- **OS-centric tiering (TPP, HeMem, MEMTIS)**: operate at millisecond granularity and cannot revoke nanosecond DMA commitments; they assume software controls residency, which an autonomous CXL 3.x endpoint invalidates.
- **Tiered orchestrators (InfiniGen, FlexGen, DistServe, Splitwise, Mooncake)**: treat the interconnect as a passive pipe with host authority over movement. Under CXL 3.x the pipe is active, the endpoint autonomous, and the host blind to concurrent state mutations.
- **Single-tier optimizers (H2O, StreamingLLM, SnapKV)**: answer what to keep in GPU memory, not whether to move data from a shared pool, and gate no hardware-committed transfer. H2O assumes the scorer can verify current-step attention mass; SnapKV requires current-step query–key interaction, violating the one-step-delay boundary.
- **Compression (KIVI, CacheGen)**: reduce per-token footprint but not the number of cross-tier descriptor submissions, a different axis than admission control.
- **Prefix caching (CacheBlend, vLLM APC)**: answer whether data *exists* in cache, not whether it should be *promoted* from a remote pool, and do not address cross-tenant redundant reads.
- **Replacement policies (LIRS, SIEVE)**: optimize hit rate assuming software-controlled, revocable eviction; under CXL 3.x commitment is hardware-driven and irrevocable.
- **Multi-tenant fairness (Equilibria, Pisces)**: assume a centralized scheduler with global visibility *before* requests enter the network; under CXL 3.x requests are committed by the host before reaching the endpoint.
- **Hardware endpoint prototypes (Samsung CMM-D, SK Hynix CMS)**: show endpoint compute is feasible but implement no admission-gate semantics; without rejection before payload issue, every speculative prefetch remains a potential RPE event.

The common thread is a conflation of where to *score* and where to *enforce*. Prior work assumes these can be decoupled; CXL 3.x proves enforcement cannot leave the endpoint, and that scoring verdicts, wherever computed, must bind at the endpoint to be effective. That is the gap PROSE closes.

Artifact and Supplementary Material Availability

This supplementary document, together with all data, scripts, and hardware sources, is available at:

<https://github.com/Anonymous-1030/PROSE-APEX>

The repository contains.

- Complete synthesizable SystemVerilog RTL (8 modules, 750+ lines), including the consult-then-CAS object directory (`rtl/cefe_directory.sv`)
- Testbenches (original 5-test suite + trace-driven cross-check), plus the directory adversary co-simulation with randomized placement updates inside the OAT window (`rtl/cefe_directory_tb.sv`)
- TLA+ consult-then-CAS specification (`formal/prose_oat_cas.tla`, `formal/prose_oat_cas.cfg`) with a Java-free exhaustive checker (`formal/check_oat_cas.py`)
- Bit-exact Python reference model
- All supplementary experiment reproduction scripts (S1–S12)
- A single-host L0 gated-pull end-to-end benchmark (`host_sw/bench_modeb_e2e.cpp`) with a fetch-then-score falsifiability control, runnable on a genuine CXL Type-3 device via `PROSE_CXL_DEVD-AX=/dev/dax0.0 ./run_modeb_hw.sh` and on an honest emulated fallback otherwise; the portable substrate backend (`cx1_devmem.h`) self-labels `real-devdax`, `real-numa-node`, or `emulated`, and records the `real_cxl` provenance flag
- The Mooncake Store RPE audit and measurement harness (`rpe_lab/`): the two-tenant replay driver, the lease-expiry discard probe and patch, the tier-A/tier-B/hard-pin configurations, and the checked-in aggregates behind the main paper’s deployed-evidence section

All supplementary experiments are reproducible via `make supplementary`. Each script generates both figures (PDF/PNG) and machine-readable JSON under `experiments/out/`.

References

- [1] Y. Wang, L. Wu, S. Gao, Y. Tang, J. Luo, Z. Wang, Y. Ou, D. Dong, N. Xiao, and M. Lai, “Cohet: A CXL-Driven Coherent Heterogeneous Computing Framework with Hardware-Calibrated Full-System Simulation,” in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2026, pp. 1–16.
- [2] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu, “BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems,” in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD)*, 2025, pp. 5831–5841.
- [3] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” in *Proc. ACM/IEEE ISCA*, 2024, pp. 118–132.
- [4] Applied Compute, “trie: Trace Replay Inference Evaluation,” GitHub repository, 2026.

- [5] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, “VAttention: Dynamic Memory Management for Serving LLMs without PagedAttention,” in *Proc. ACM ASPLOS*, 2025, pp. 1133–1150.
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. ACM SOSP*, 2023, pp. 611–626.

Table 1: Artifact map for all reported experiments. Paths are relative to the artifact repository root. Runtime classes: closed-form SimCXL <2 min each, RTL simulation minutes, Mooncake deployment replay hours.

Paper item	Script	Seeds	Checked-in output	Runtime
Optimistic-Reclaim table	experiments/run_optimistic_reclaim.py, run_oversub_low.py	10	experiments/out/optimistic_reclaim*/	<2 min
Design-space table	experiments/run_design_space.py	0–9	results/design_space.json	<2 min
Enabler gate×autonomy table	experiments/run_enabler_matrix.py	0–9	results/enabler_matrix.json	<2 min
Placement-isolation table	experiments/run_placement_isolation.py	n/a	results/placement_isolation_full.json	<2 min
L0 gated-pull harness	host_sw/bench_modeb_e2e.cpp via host_sw/run_modeb_hw.sh	n/a	experiments/out/ modeb_hw/ (self-labelled real_cxl: false)	minutes
Stale-byte harm figure	experiments/run_rpe_harm.py	2,000 trials/point	results/rpe_harm.json	<2 min
Cross-tenant leak PoC	experiments/run_cross_tenant_leak.py	n/a	results/cross_tenant_leak_poc.{json,txt}	<2 min
Mechanism comparison figure	experiments/baselines/run_baseline_sweep.py	paired	results/baselines/	<2 min
GenOnly+epoch-fence baseline (16.4–16.6% stale)	experiments/run_design_space_epochfence.py, experiments/baselines/run_epochfence_sweep.py	0–9	results/design_space_epochfence.json	<2 min
Mechanism ablation figure	experiments/run_mechanism_ablation.py	n/a	experiments/out/mechanism_ablation/	<2 min
Phase-diagram figure	experiments/run_phase_diagram.py	n/a	experiments/out/data/phase_diagram.json	<2 min
Pin-pool crossover (§S7)	experiments/run_pin_crossover.py	n/a	results/pin_crossover.json	<2 min
Host-prescore (375/500 tok/s)	sweep experiments/run_host_prescore.py	n/a	experiments/out/host_prescore/	<2 min
Hybrid (78.2 tok/s), off 2.58×	validator trace_adapter/cfo_coalescing_analysis.py	n/a	results/cfo_overlap_analysis.json	<2 min
Multi-host P99 (292 ns)	experiments/run_simcxl_multihost.py	10 trials	results/multihost_p99.json	<2 min
Single-host (377.0/377.4 tok/s)	parity experiments/run_single_host_parity.py	42, 123, 2024, 9001	results/single_host_parity.json	<2 min
Directory sizing	experiments/run_directory_sizing.py	n/a	results/directory_sizing.json	<2 min
Rejection-cost decomposition	experiments/run_reject_cost.py	0–9	results/reject_cost.json	<2 min
Byte-conservation envelope [3.13, 3.60]×	experiments/analytical_bound.py	closed form	experiments/out/data/analytical_bound.json	<2 min
SLO goodput conversion (§S7)	experiments/run_slo_goodput.py	0–9	experiments/out/slo_goodput/slo_goodput.json	<2 min
Page/block-cache generality (§S12)	experiments/run_page_cache_generality.py	n/a	experiments/out/page_cache/	<2 min
Mooncake tier-A accounting (32,908 fires / 81,649 reads, 32,983 discards, 120.8 GB)	rpe_lab/analysis/aggregate.py	42	rpe_lab/results/tier*.json	hours

Table 2: The seven OAT actions in execution order.

Step	Information read	Reject condition	Safety role
1. check capability	namespace, capability bits	capability invalid	foreign descriptors stay out
2. check generation	object directory	g behind directory	incarnation is current
3. check extents	slot-to-incarnation map	extent not mapping to $\langle id, g \rangle$	slot still holds the object
4. check state	replay, abort state	duplicate or aborted	one transfer per admission
5. check resources	pin table, queues	no free pin entry	the hold can be recorded
6. acquire hold	pin table	n/a	binding survives the transfer
7. mint token	token store	n/a	completion maps to this admission

Table 3: Area breakdown of the optional APEX scoring pipeline (ASAP7 7 nm, 1 GHz), from the structural cell-area estimate in `asic/reports/area.rpt`. Cell area totals $18,761 \mu\text{m}^2$; a $1.3\times$ routing/filler/power-grid factor yields $24,390 \mu\text{m}^2$ ($=0.024 \text{ mm}^2$). The hardware provisions 7 expert banks for runtime reconfigurability; Core2 activates 2 and clock-gates 5.

Component	Cell area (μm^2)	%
Expert Banks ($7 \times 512 \times 16\text{b DFF RF, 1R1W}$)	17,193	91.6
Top-K Selection ($K=25$, min-heap + 17-cmp min-tree + arrays)	368	2.0
MAC Array ($7 \times 8\text{b} \times 16\text{b mult} + 4\text{-level CSA} + \text{prefix CPA}$)	411	2.2
Identity/residency unit (512-bit residency vector + comparators)	166	0.9
Weight Update (64-entry LUT + 7 mult + normalize)	290	1.5
Loss Compute + exploration probe	216	1.2
Pipeline Controller + Top + ICG	117	0.6
Total cell area	18,761	100.0
With $1.3\times$ routing/filler/grid	24,390	n/a
Scoring pipeline area (7-bank provisioned):		
Expert bank state (7 banks $\times$ 512 $\times$ 16b):	7 KiB (DFF)	
Full endpoint (projection, §S4):	1.069 mm^2	
Full endpoint bulk state (~ 209 KiB SRAM + 7 KiB DFF):	~ 216 KiB total	
Scoring pipeline power:	~ 12.6 mW (3.0 mW dynamic + 9.6 mW leakage)	
Full endpoint power (16-host sustained load):	~ 78 mW	

Table 6: Per-block power density, pre-layout estimates at switching activity $\alpha=0.15$ (block areas from the block-level projection of §S4). Block powers are worst-case corner values used for IR-drop sign-off and therefore do not sum to the nominal pipeline power of §S3.

Block	Area (mm^2)	Power (mW)	Density (mW/mm^2)	Risk
Expert Banks ($7\times$)	0.0750	9.30	124.0	LOW
MAC Array	0.0080	2.30	287.5	LOW
Top-K Heap	0.0090	1.80	200.0	LOW
Weight Update	0.0040	0.75	187.5	LOW
Identity/residency logic	0.0020	0.32	160.0	LOW
Exploration probe + LFSR	0.0010	0.08	80.0	LOW
VC-WRR Arbiter	0.0450	4.30	95.6	LOW
BDB Parser	0.0150	1.30	86.7	LOW

Table 7: Area decomposition of the shared endpoint logic (ASAP7 7 nm, 1 GHz). With the 0.024 mm² scoring pipeline, the full endpoint totals 1.069 mm². This is a block-level *projection*, not a flat synthesis result.

Component	Area (mm ²)	%
VC-WRR Arbiter + Per-Tenant Queue State (16 VCs $\times$ 8-entry SRAM queues + priority logic)	0.180	17.2
Coalescing Matcher (CAM) (16-entry CAM + fan-out steering + handle compare)	0.120	11.5
Capability Validation Unit (64-bit digest comparator + per-session token regs)	0.090	8.6
BDB Parser + DMA Engine Control (descriptor decode + doorbell + completion ring)	0.220	21.1
SRAM Macro State ($\sim$ 209 KiB: VC buffers, BDB staging, directory, validation)	0.310	29.7
Routing, Power Grid, Analog/IO	0.125	12.0
Total (shared endpoint logic)	1.045	100.0

Table 8: Structural logic and state breakdown at equal synthesis fidelity. Area/power are ASAP7 structural estimates excluding SRAM macros and replicated 16-host staging; state reports RTL-declared bits. The rows report only the synthesized register-file netlist and are not a decomposition of the 1.069 mm² projection. Estimates carry $\approx \pm 30\%$ structural uncertainty.

Group (modules)	Area (mm ²)	Power (mW)	State (KiB)
OAT safety path (identity/residency check, pin table, descriptor parser)	0.0061	5.83	1.58
Multi-tenant scheduling (VC-WRR fairness arbiter)	0.0251	18.28	8.06
Optional ranking/coalescing (scorer, top- K , experts, update, probe, coalescing CAM)	0.0251	13.33	7.58
Shared pipeline glue	0.0002	0.14	0.03
Structural total	0.0565	37.58	17.25

Table 13: Unmitigated RPE_{payload} across public inference traces and eviction policies (16 hosts, $1\times$ capacity), and the endpoint-gated result. Values reproduce the main paper’s headline (BurstGPT 12.8%, Splitwise 14.4%, trie 11.2% under LRU) and show the <0.3 pp per-policy spread.

Trace	LRU	ARC	LIRS	SIEVE	Gated
BurstGPT [2]	12.8%	12.7%	12.9%	12.6%	0%
Splitwise Conv [3]	14.4%	14.3%	14.5%	14.2%	0%
trie (agentic) [4]	11.2%	11.0%	11.3%	11.1%	0%

Table 14: Mapping of the main paper’s zero-stale-payload theorem assumptions to evaluated single-extent KV RTL coverage.

Assumption	RTL coverage
(a) Atomic validate+hold	OAT fuses directory lookup, generation match, and pin increment
(b) Payload after success	Payload gate opens only on an ADMITTED verdict
(c) Hold until completion	Per-transfer pin released at DMA completion or abort
(d) Reclaim at zero pin	Eviction/frame reuse checks the pin table before overwrite

Table 15: Two-phase reservation table sensitivity (16 hosts, 2 GB/s/tenant). Overrun α is the oversubscription at which in-flight count exceeds table capacity; P99 is at $32\times$. The OAT holds no reservation across a round-trip and stays flat. End-to-end chunk P99 covers queue wait, reservation hold where the mechanism takes one, and full payload service; it is the SLO-relevant boundary, and it differs from the admission P99 of the main paper (292 ns), which measures only the endpoint gate and excludes queueing and payload.

Table entries	Area (KiB)	Overrun α	End-to-end chunk P99 @ $32\times$	RPE
256	16	$4\times$	$1,642\ \mu\text{s}$	0%
512	32	$8\times$	$1,589\ \mu\text{s}$	0%
1024	64	$16\times$	$1,520\ \mu\text{s}$	0%
2048	128	$24\times$	$1,483\ \mu\text{s}$	0%
4096	256	$48\times$	$1,401\ \mu\text{s}$	0%
OAT (PROSE)	0 (no table)	n/a	$1,057\ \mu\text{s}$	0%

Table 16: Full placement-isolation sweep. Cred derives from the calibrated host-credit decomposition; EP is zero by construction. Artifact: `experiments/run_placement_isolation.py`.

Hosts	Stale (%)			Jain			MisBW (MB)		
	Opt	Cred	EP	Opt	Cred	EP	Opt	Cred	EP
1	0.00	0.00	0.00	1.000	1.000	1.000	0	0	0
2	34.24	7.20	0.00	0.999	1.000	1.000	323	73.7	0
4	32.39	10.80	0.00	0.992	1.000	1.000	1,745	110.6	0
8	29.57	12.60	0.00	0.964	1.000	1.000	4,745	129.0	0
16	38.86	13.50	0.00	0.920	1.000	1.000	8,994	138.2	0

Table 17: L1 platform prerequisites, evidence levels, and fallbacks.

Prerequisite	Spec basis	Status	Fallback
ACS forces P2P via IOMMU	PCIe ACS	platform-dependent	fail closed to L0
PASID on peer writes	PCIe PASID	platform-dependent	per-tenant BAR window
IOMMU translation of P2P	VT-d / SMMU	platform-dependent	fail closed to L0
BAR mapping, system scope	PCIe BAR	platform-dependent	fail closed to L0
Dual-root P2P routing	topology	platform-dependent	host-bounce path
GPU visibility at L2	none	unmeasured	L1 remains conditional

Table 18: Same-contract Recovery@K across shift regimes.

Scorer	Stable	Moderate	Frequent	Adversarial
<i>Deployment-adapted baselines</i>				
APEX-Core2 (ours)	0.918	0.948	0.921	0.879
InfiniGen-CXL	0.856	0.925	0.896	0.858
Δ (pp)	+6.2	+2.3	+2.5	+2.1
Causal-GRU	0.561	0.532	0.402	0.298
APEX-fixed	0.832	0.598	0.554	0.523
Quest-CXL	0.319	0.325	0.319	0.297
<i>Native oracle, non-deployable (violates the one-step-delay boundary)</i>				
Native-InfiniGen	0.938	0.976	0.974	0.977

Table 19: Recovery@K and first-observation latency under extreme domain shifts (400 steps, 4 abrupt transitions, zero inter-domain overlap).

Scorer	Mean Recovery@K	Obs. Latency (steps)
APEX-Core2	0.412	1.0
APEX-Core2 + probe	0.405	1.0
Causal-GRU	0.032	>20
Random baseline	0.117	n/a

Table 21: Coalescing bandwidth saving under flat (16-entry CAM only) and hierarchical (CAM + 256-entry switch directory) architectures across host counts.

Hosts	Switches	Flat	Hierarchical
4	1	13.0%	13.0%
8	1	15.1%	15.1%
16	1	16.2%	16.2%
32	2	16.8%	23.0%
48	3	16.9%	25.2%
64	4	17.0%	26.3%

Table 22: Recovery@K degradation under increasing context length, measured over the *full logical address space* including long-cold chunks (deliberately pessimistic; the headline evaluation of §S5 reports *active-set* Recovery@K over the 200–400 chunks whose attention exceeds threshold). The 512-entry Expert Bank overflows once the logical space exceeds 512 chunks.

Context	Chunks	Overflow	Mean Recovery@K	Collision Rate
64K	1,024	Yes	0.334	0.656
128K	2,048	Yes	0.371	0.663
256K	4,096	Yes	0.305	0.673
512K	8,192	Yes	0.187	0.704
1M	16,384	Yes	0.103	0.721

Table 24: Cross-model task accuracy on the *enlarged* suite (primary 7 workloads + 128K retrieval, code, summarization), normalized to FullKV. The harder suite lowers absolute accuracy relative to the main paper’s 7-workload operating-point figures. At 50% budget the Qwen2.5 models cross 0.80; LLaMA-3.1-8B (0.791) and Mistral-7B (0.778) fall just short.

Model	5%	10%	15%	20%	50%	Worst task@5%	Worst trial@5%
Qwen2.5-7B	0.237	0.329	0.402	0.465	0.812	0.102 (RULER-VT)	0.088
LLaMA-3.1-8B	0.218	0.341	0.421	0.483	0.791	0.071 (Adv.)	0.012
Mistral-7B	0.196	0.302	0.379	0.440	0.778	0.054 (Multi-Doc)	0.000
Qwen2.5-14B	0.264	0.372	0.448	0.510	0.824	0.131 (RAG)	0.105

Table 27: Potentially applicable mapping of the Commit-Time Object Contract to a page/block cache (design analysis, not measured).

KV instance	Page/block-cache instance
chunk generation g	page epoch
pool slot	cache frame
promotion descriptor	prefetch/read descriptor
chunk eviction	frame reuse